OS NOTES

A **load/store instruction** goes onto the bus with:

- an address
- a read/write signal
- data (for stores)

1. Multiplexing

Multiplexing = controlled sharing of a physical resource

Two types:

- **Time multiplexing** → CPU
- **Space multiplexing** → memory, disk blocks

CPU multiplexing

CPU executes:

- process A for a while
- process B for a while
- process C for a while

Hardware help:

- timer interrupt forces control back to OS

Without this:

- first program to run never stops

Device multiplexing

Only the kernel:

- touches device registers
- queues requests
- serializes access

User programs:

- submit requests
- never touch registers

2. Abstraction

“You have a CPU” → process
“You have memory” → virtual address space
“You have a disk” → files
“You have a network” → sockets

Process = running program with private state

This includes:

- PC
- registers
- stack

- heap
- file descriptor table

3. Protection

A program could:

- overwrite kernel memory
- disable interrupts
- reprogram disk controller
- spy on other processes
- who can execute which instructions
- who can access which addresses

A **mode** is a CPU state bit that changes:

- which instructions are legal
- which addresses are accessible

On RISC-V:

- User mode (U)
- Supervisor mode (S)

If user-mode code tries to:

- write page tables
- touch device registers
- disable interrupts

CPU raises a trap

If all processes use physical addresses, they collide

Solution: Address translation

CPU now interprets memory as:

```
(Virtual Address, ASID) → Physical Address
```

Each process:

- sees VA 0x4000
- but maps to different PA

User programs:

- cannot touch devices
- cannot manage memory mappings
- cannot create processes directly

Syscall = controlled entry to kernel

User program:

1. Puts syscall number in register (a7)
2. Puts arguments in registers (a0–a6)
3. Executes `ecall`

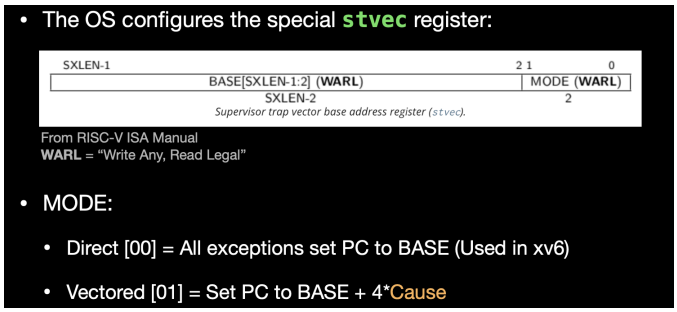
OS NOTES

CPU:

- saves PC
- switches to supervisor mode
- jumps to OS handler

• Saves all registers
• Invokes the appropriate function based on syscall number (i.e., a7 value)
• OS maintains a mapping (“syscall table”) between number and function
• The result is stored in a0
• Restores all registers
• Invokes the sret instruction
• Restores the previous mode, SP, and PC (i.e., returning to user execution)

• **How does the processor know the OS syscall handler address?** When a trap happens while running in user mode (or supervisor mode), where should the CPU jump to start handling it? Because on a trap, you need a trusted place to jump to. If user code could choose where to jump, user code could fake being the OS. So hardware says: only privileged code (the OS) can set `stvec`, and the CPU will always go there on a trap.



`a7` contains syscall number; `a0–a6` contain arguments

`ecall` causes the CPU to:

- save caller PC
- save caller mode
- set mode to supervisor
- set PC to supervisor’s syscall handler address

You are in **user mode** executing some instruction stream. Registers look like:

- `a7 = SYS_write` (for example)
- `a0 = fd`
- `a1 = buf_ptr`
- `a2 = len…`

Record where you were (save the current PC somewhere like `sepc`)
Record why you trapped (`scause = environment call from U-mode` for a syscall)
Switch privilege: User → Supervisor
Jump to the trap entry address by loading `PC` from `stvec` (direct mode means just `BASE`)
OS saves state and executes the syscall handler based on `a7`

OS handler “saves all registers”, dispatches using a syscall table, stores result in `a0`, restores regs, then does `sret`

Look in xv6:

- `kernel/trampoline.S – uservec`
- `kernel/trap.c – usertrap() and usertrapret()`

- `kernel/syscall.c – syscall()`

Layer 0: `stvec` points to an entry stub (assembly)

In xv6, this entry stub is in `trampoline.S` (`uservec`)

Job of this layer: do the minimal machine-critical steps:

- switch to a known safe stack / environment
- save registers somewhere (trapframe)
- then call into C code

Layer 1: `usertrap()` (C-level trap dispatcher)

This is in `kernel/trap.c`

Job: look at trap cause and decide:

- Syscall? exception? interrupt?

Layer 2: `syscall()` (syscall dispatcher)

This is in `kernel/syscall.c`

Job: use `a7` as an index into a syscall table:

- mapping number → function pointer

Then:

- call the right kernel function
- put return value into `a0`

So when people say “syscall handler,” they might mean:

- the trap entry (`uservec`)
- or the trap dispatcher (`usertrap`)
- or the syscall table dispatcher (`syscall()`)
- or the individual syscall implementation (`sys_read`, `sys_fork`, …)

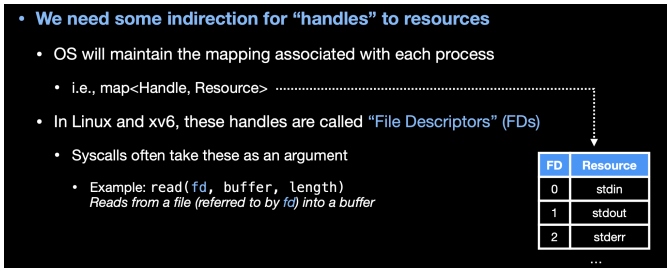
They’re all part of the path.

5.2 What state must be prepared before `sret`

The kernel must set things up so that:

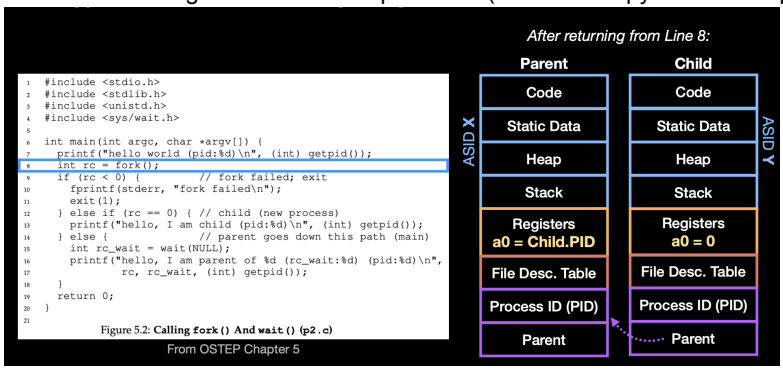
- `sepc` (saved PC) points to the next user instruction after `ecall`
- the status register indicates the next mode should be user
- registers are restored so the user program continues normally

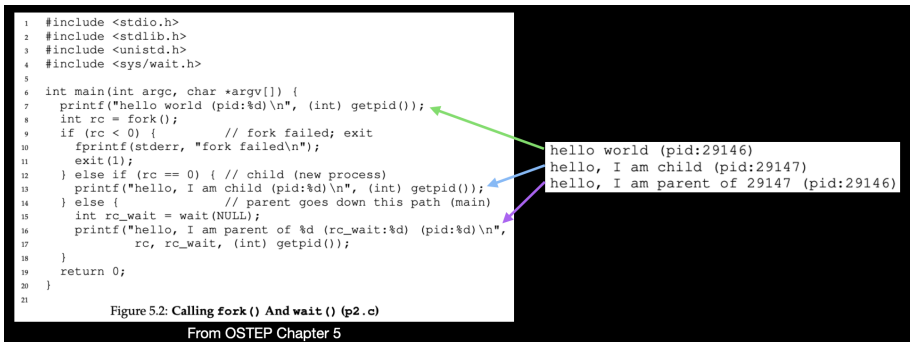
OS NOTES

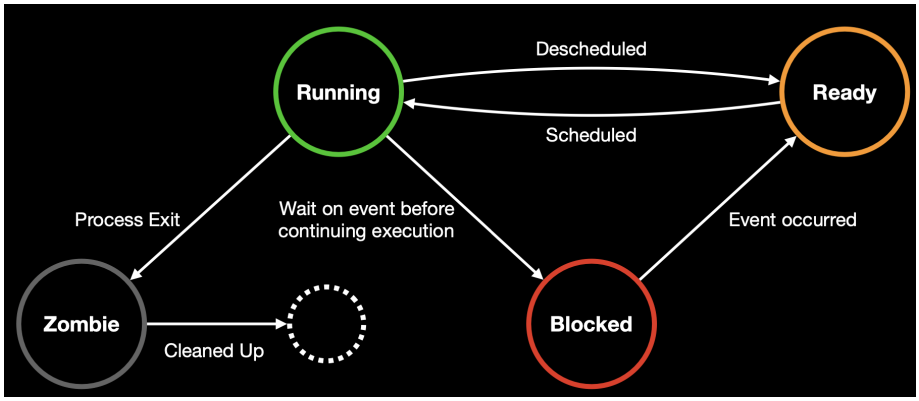


fork( ) → Process ID (PID)

- **Creates a “fork” of the calling process**
- Essentially, an (*almost*) exact copy of the process
- Calling process is the “parent”, new process is the “child”
- **What is different?**
- The child is assigned a new PID
- The child is assigned a new address space (but has a copy of the memory)
- The child is assigned a new descriptor table (but has a copy of all descriptors)







When a child exits, it usually provides a small integer status code (like `exit(0)` for success).

- `wait(&status)` lets the kernel write that status into the parent’s memory.

- If you pass `NULL`, you're saying "I don't care about the status."

Because the kernel can't return two values in the syscall return register (only one). So: one value comes back as the return value (child PID), the other is written into a user-provided memory location.

This is a common syscall pattern:

- return one value in `a0`
- write additional outputs into user memory pointed to by args

Blocking means:

> the parent stops running and enters a "sleeping"/blocked state inside the kernel until the event occurs.

Your process-state slides earlier show the blocked state as "wait on event before continuing execution". parent calls `wait()` → kernel checks "do I already have a zombie child?"

- if yes: return immediately
- if no: put parent to sleep on a "wait channel"

child calls `exit()` → kernel:

- marks child as exited (zombie)
- stores exit status
- wakes up parent if it is sleeping in `wait()`

After `fork()` returns, the kernel may schedule:

- parent first
- child first
- or interleave them

So if both do `printf("hi\n")` you can get:

- parent then child
- child then parent
- or even mixed output (if prints are not atomic)

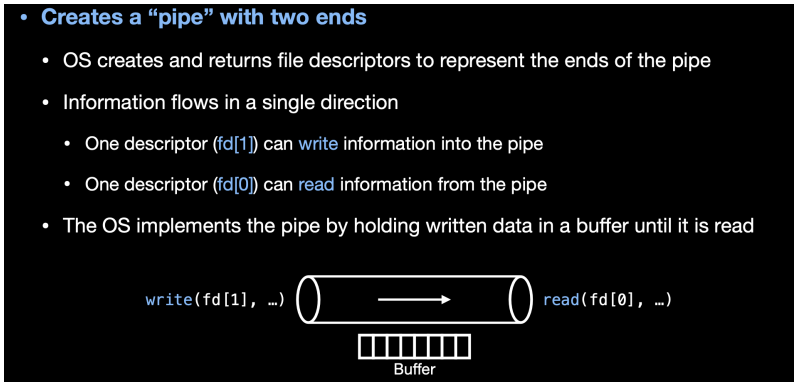
The slide is probing whether you understand:

> concurrency ⇒ execution order depends on scheduling, interrupts, timing

The key point: there is nothing in the semantics of `fork()` that says "parent goes first".

2.3 How wait changes this

If parent calls `wait()` after fork, the parent will block until the child exits (as explained above). That creates *some* ordering constraint (parent won't pass the wait until child exits), but it still doesn't force which one prints *before* the wait call happens unless you structure code that way.



In Unix these capabilities are represented as file descriptors:

- `fd[0]` = read end
- `fd[1]` = write end

3.2 “OS creates and returns file descriptors”

A file descriptor is:

- an integer in the process’s descriptor table
- that points to an underlying kernel “open file” object (which points to a pipe object)

Your lecture explicitly frames this as:

- OS maintains mapping `map<Handle, Resource>`
- in Linux/xv6 handles are file descriptors

So `pipe(fd)` means the OS:

1. allocates a new pipe object (buffer + metadata)
2. allocates two descriptor entries in your process:
    - one marked readable → points to pipe
    - one marked writable → points to pipe

3.3 “Information flows in a single direction”

This means:

- bytes written to write end become readable at read end
- there is no built-in reverse direction

So if you want two-way communication, you need **two pipes** (one for each direction).

When you call:

- `write(fd[1], data)` → kernel copies bytes into pipe buffer
- `read(fd[0], buf)` → kernel copies bytes out of pipe buffer into your memory

If buffer is empty:

- reads may block (until data arrives or all writers are closed)
  If buffer is full:
- writes may block (until readers consume)

4) IPC with pipes — parent→child and two-way

Slide asks:

- parent wants to write to child: how?
- extend to two-way communication: how?

4.1 Parent writes to child (one pipe)

Plan:

1. parent creates pipe **before** fork
2. fork duplicates the file descriptor table into child (so child also has those fds)
3. then:
    - parent closes read end and writes
    - child closes write end and reads

Why before fork?
Because after fork, the parent and child are separate processes. Creating the pipe before fork guarantees both inherit the endpoints.

4.2 Two-way communication (two pipes)

You need:

- pipe1: parent → child
- pipe2: child → parent
- parent writes into pipe1's write end; child reads from pipe1's read end
- child writes into pipe2's write end; parent reads from pipe2's read end

And you must close the unused ends in each process, or EOF/blocking gets messed up.

Your process has an FD table. Each FD entry points to a kernel object representing an "open file description".

When you `dup(oldFD)`:

- you do *not* copy the file or pipe
- you create a new FD table entry that points to the **same** kernel object

So now you have two different integers, same resource.

Example mental model:

- FD 1 and FD 7 both point to the same pipe write end.
  Writing to either writes into the same pipe.

5.2 `dup(oldFD)` chooses "lowest available"

Why? Because Unix traditionally allocates the smallest unused fd number. This is convenient and predictable.

5.3 `dup2(oldFD, newFD)` is the “surgical” version

Sometimes you need a **specific** fd number (especially 0,1,2). That is literally the use case: redirecting stdin/stdout/stderr.

`dup2` rule:

- if `newFD` is already open, close it first
- then make `newFD` refer to same underlying open file object as `oldFD`

So after `dup2(p[1], 1)`, fd 1 (stdout) now points to the pipe write end.

That’s shell redirection and pipelines.

int p[2];

pipe(p);

close(1);

dup(p[1]);

printf("Hello Pipe!\n");

6.1 Before anything: base FD table

Typical at program start:

- fd 0 = stdin

- fd 1 = stdout

- fd 2 = stderr

6.2 After `pipe(p)`

The OS allocates two new FDs, usually the next available numbers, e.g. 3 and 4.

E.g. int p[2];

pipe(p); // p[0] = read end, p[1] = write end

So:

- `p[0]` might be 3 (read end)

- `p[1]` might be 4 (write end)

Now:

- fd 3 → pipe read end

- fd 4 → pipe write end

6.3 After `close(1)`

You close stdout.

Now fd 1 is free.

6.4 After `dup(p[1])`

`dup` allocates the **lowest free fd number**.

But we just freed fd 1. So the lowest free is 1.

So `dup(p[1])` returns 1, and fd 1 now points to the same underlying resource as `p[1]` (pipe write end).
:contentReference[oaicite:17]{index=17}

So now:

- fd 1 → pipe write end

- fd 4 → pipe write end (still)

(two FDs to same resource)

6.5 Then `printf("Hello Pipe!\n");`

Where does printf write?

Ultimately, printf writes to fd 1 (stdout).

But fd 1 is no longer the terminal: it's the pipe write end.

So those bytes go into the pipe buffer instead of the screen.

That is the whole trick behind shell redirection/pipelines:

- change what fd 0/1/2 refer to

- then `exec` a program

- program thinks it's writing to stdout, but stdout is wired elsewhere

---

7) `exec()` — what it means to "change the program being executed"

Slide says:

- "Changes the program being executed by the current process"

- "Can supply arguments via argv"

- "Common pattern: fork() then exec(…)"

- fork creates child

- exec replaces child with program+args

- example: shell :contentReference[oaicite:18]{index=18}

Now the full explanation.

OS NOTES

7.1 What does “same process, different program” mean?

A process has:

- PID (identity)

- resources (file descriptors, etc.)

- an address space (memory mappings)

- CPU state when running

`exec(...)` does **not** create a new process.

Instead, it says:

> “Throw away the current program image (code/data/heap/stack) and load a new one into this *same* process.”

So after exec:

- PID is the same

- but the code running is completely different

7.2 What happens to memory?

The old address space is replaced:

- old code/data/heap/stack are discarded

- new program is loaded into memory

- stack is built with argv strings, argc, etc.

That’s why exec is “replace”.

7.3 What happens to file descriptors?

Crucial: file descriptors generally remain open across exec (unless marked close-on-exec in full Unix; xv6 keeps it simple).

This is **why** shells can set up pipes/redirection **before** exec:

- shell calls pipe/dup/close to configure fd 0/1

- then exec child into `cat` or `head`

- the new program inherits the configured fd table

This is exactly what your pipeline slide says: “shell configures the file descriptors for each process prior to invoking exec”

7.4 Why “fork then exec” is the shell pattern

The shell wants to run a command *without dying itself*.

If the shell just called exec directly, the shell would be replaced by the program (you’d lose the shell).

So the shell:

1) `fork()` to create child

2) child does setup (pipes, redirection)

3) child calls `exec(cmd, argv)` to become the command

4) parent shell calls `wait()` to collect child, then continues prompting

Your slide literally says “Common Pattern: fork() then exec(…)” and “Example: The Shell!” :contentReference[oaicite:20]{index=20}

If you reply **continue** again, next I will do:

8) Putting it all together: `cat LICENSE | head -c 16`

At the same depth, we will walk through:

- exactly which process does which syscall in what order

- exactly how fd tables look in each process

- why cat’s stdout becomes pipe(W) and head’s stdin becomes pipe(R) :contentReference[oaicite:21]{index=21}

- how closing endpoints controls EOF and prevents deadlock

OS NOTES

9) Then the “mode switch” part

- traps are synchronous user→kernel switches

- interrupts are async wires with IDs

- PLIC claim/complete registers and why MMIO matters

- upcalls/signals and why they *cannot* be “user→user function call”

cat LICENSE | head -c 16

means (in English):

Run two programs at the same time

Take everything cat writes to stdout

Feed it as stdin into head

How can the OS make one program’s output become another program’s input without either program knowing?

The shell’s job is **NOT**:

- “run cat”
- “run head”

The shell’s job is:

> **Create processes whose file descriptor tables are wired together correctly, THEN start the programs.**

the shell does:

fork()

→ child

change FD table

exec("cat")

and

fork() → child change FD table exec("head")

→ child

change FD table

exec("head")

CAT LICENSE | HEAD -C 16 : FULL CODE-TO-EFFECT EXPLANATION

This document explains, line by line, exactly which part of the shell code

does what when implementing the pipeline:

cat LICENSE | head -c 16

There is NO abstraction or skipping. Each syscall is mapped directly

to its effect on the file descriptor table.

---

1. CREATE THE PIPE

int p[2];

pipe(p);

Effect:

- Kernel allocates ONE pipe object (buffer + metadata)

- Returns two file descriptors:

p[0] = Pipe read end

p[1] = Pipe write end

Shell FD table:

0 -> stdin

1 -> stdout

2 -> stderr

3 -> Pipe(R)

4 -> Pipe(W)

---

2. FORK CHILD FOR CAT

if (fork() == 0) {

Effect:

- Child process is created

- Child inherits a COPY of the FD table

Child FD table:

0 -> stdin

1 -> stdout

2 -> stderr

3 -> Pipe(R)

4 -> Pipe(W)

---

3. REDIRECT STDOUT TO PIPE (CAT)

close(1);

Effect:

- FD 1 (stdout) is freed

FD table:

0 -> stdin

1 -> FREE

2 -> stderr

3 -> Pipe(R)

4 -> Pipe(W)

---

dup(p[1]);

Effect:

- Duplicates Pipe(W)

- Assigned to lowest free FD (1)

FD table:

0 -> stdin

1 -> Pipe(W)

2 -> stderr

3 -> Pipe(R)

4 -> Pipe(W)

THIS IS WHY CAT WRITES INTO THE PIPE.

---

close(p[0]);

close(p[1]);

Effect:

OS NOTES

- Cat does not read from pipe

- Extra write FD is removed

FD table:

0 -> stdin

1 -> Pipe(W)

2 -> stderr

---

exec("cat", ["cat", "LICENSE"]);

Effect:

- Program image replaced with cat

- FD table preserved

Cat behavior:

read(0) -> stdin

write(1) -> Pipe(W)

---

4. FORK CHILD FOR HEAD

if (fork() == 0) {

Effect:

- New child process created

- Inherits shell FD table

FD table:

0 -> stdin

1 -> stdout

2 -> stderr

3 -> Pipe(R)

4 -> Pipe(W)

---

5. REDIRECT STDIN TO PIPE (HEAD)

close(0);

Effect:

- FD 0 (stdin) freed

FD table:

0 -> FREE

1 -> stdout

2 -> stderr

3 -> Pipe(R)

4 -> Pipe(W)

---

dup(p[0]);

Effect:

- Pipe(R) duplicated into FD 0

FD table:

0 -> Pipe(R)

1 -> stdout

2 -> stderr

3 -> Pipe(R)

4 -> Pipe(W)

THIS IS WHY HEAD READS FROM THE PIPE.

---

close(p[1]);

close(p[0]);

Effect:

- Head does not write to pipe

- Extra read FD removed

FD table:

0 -> Pipe(R)

1 -> stdout

2 -> stderr

OS NOTES

---

exec("head", ["head", "-c", "16"]);

Effect:

- Program image replaced with head

- FD table preserved

Head behavior:

read(0) -> Pipe(R)

write(1) -> stdout

---

6. SHELL CLEANUP

close(p[0]);

close(p[1]);

Effect:

- Shell releases its copies of pipe FDs

- Ensures correct EOF behavior

---

7. WAIT FOR CHILDREN

wait();

wait();

Effect:

- Shell blocks until both cat and head exit

- Prevents zombie processes

FINAL RESULT:

- cat writes bytes into Pipe(W)

- kernel buffers bytes

- head reads bytes from Pipe(R)

- only first 16 bytes are printed

- neither program knows a pipe exists

OS NOTES

ALL POWER COMES FROM FILE DESCRIPTOR REWIRING.**The key trick: set up FDs BEFORE exec**

Because open FDs remain open across exec (that's why redirection works): OSTEP explicitly says you can close stdout then open/redirect before exec and the new program's output goes to the new place

---

A **trap** is a CPU-enforced control transfer:

- you were executing in user mode (or supervisor mode)
- some event occurs (ecall / illegal instruction / timer / device)
- the CPU **forces**:
    1. switch to supervisor mode
    2. jump to a kernel entry point (from `stvec` in RISC-V)
    3. record *why* it trapped and *where* it trapped from

The key idea: **the user program does not control the destination**. Hardware does.

Your notes say syscalls are **synchronous** and are a forced transfer of control to the kernel

Meaning of synchronous:
It happens *exactly* at a particular instruction boundary because your code executed the syscall instruction. If you run the same program, you trap at the same point (assuming same control flow).

These are "the current instruction could not be completed safely/legally."

Examples:

- illegal instruction
- divide by zero
- page fault (virtual memory translation fails or permissions fail)

They are **synchronous** because they are triggered by the *current* instruction.

What kernel does depends on cause:

- kill process (illegal instruction)
- fix situation and resume (page fault handler, copy-on-write, demand paging)

Your notes emphasize interrupts are **asynchronous external events** and can occur "at any point"

Meaning of asynchronous:
An interrupt is not caused by "the instruction you're executing" in the logical sense. It can arrive between instructions because:

- timer tick occurred
- disk finished an I/O
- keyboard input arrived

This is what makes interrupts the basis of:

- preemption (timer interrupt forces kernel to regain CPU)
- device completion notification (I/O done)

2) “Ideal interrupt handling sequence” (save → handle → restore)

2.1 “Switch mode”

Hardware must force kernel mode because handling interrupts requires privileged operations:

- reading device controller state
- acknowledging interrupts
- possibly scheduling another process

So first step is privilege escalation: user→supervisor.

2.2 “Save context”

Context = everything needed to resume exactly where you left off, as if interrupt never happened.

At minimum:

- program counter (where to continue)
- registers (because handler will clobber them)

In xv6-style designs, the kernel saves this in a **trapframe** associated with the current process.

2.3 “Do work”

Work depends on interrupt source:

- timer: update ticks, maybe preempt and schedule
- disk: mark I/O complete, wake sleeping processes
- console: read input, wake readers

This is where the kernel turns a raw hardware event into a high-level effect.

2.4 “Restore context + mode”

When done:

- restore the saved user registers/PC
- return to user mode (via `sret` on RISC-V)

If the kernel decides to schedule another process, it restores *another* process’s context instead

3) PLIC: how external interrupts are delivered (RISC-V)

3.1 What is the PLIC?

PLIC = **Platform-Level Interrupt Controller**.

It sits between devices and the CPU and handles:

- prioritization (which interrupt matters most)
- routing (which CPU core/hart receives it)
- a “claim/complete” handshake so interrupts aren’t lost or repeated incorrectly

3.2 Why do we need an interrupt controller at all?

Because many devices can interrupt:

- disk
- network
- UART
- timer sources, etc.

If the CPU had to directly wire and prioritize all of them itself, the platform becomes messy. PLIC centralizes it.

3.3 How kernel talks to PLIC (MMIO)

“Accessed through MMIO” means:

- PLIC has registers mapped into physical address space
- kernel reads/writes those addresses to talk to it

So: reading a memory address is actually reading the PLIC’s internal hardware register.

3.4 Claim/Complete protocol (every step matters)

Your notes list the protocol explicitly

1. **Claim**: CPU reads PLIC “claim” register → gets an interrupt ID
    - This ID identifies which device raised the interrupt. ID corresponds to the highest-priority interrupt
2. **Handle**: kernel dispatches based on ID
    - e.g., if ID corresponds to UART, call UART driver interrupt routine.
3. **Complete**: kernel writes the same ID back to PLIC “complete” register
    - tells PLIC: “I’ve handled it; you may deliver the next one.”

If you don’t complete: you can get stuck (PLIC keeps thinking it’s pending) or you won’t receive further interrupts of that type.

3.5 “Interrupts during interrupt handling?”

Your notes say typical single-core behavior is:

- handler runs with interrupts disabled
- re-enabled after
- controlled by `sstatus.SIE`

Meaning: while kernel is in the middle of handling one interrupt, it usually does not want to be re-entered unexpectedly (would complicate correctness), so it disables further interrupts temporarily.

Upcalls are asynchronous notifications from kernel-to-user
Upcalls don’t directly alter the execution of the process
• i.e., you can not have User → User (Signal Handler) like a function call
• Kernel delivers signals when the process is scheduled
• If signal is pending, PC should be <signal handler address>
• Otherwise, PC should be the next instruction for the process
• Kernel maintains two different execution contexts
• One for the process and a new one for the signal handler

Core question of scheduling

> *Given many runnable tasks and limited CPU cores, which task should run next, and for how long?*

Schedulers are policies, not mechanisms:

- Mechanisms: context switch, timer interrupt, sleep/wakeup
- Policy: who runs next, priority, fairness, responsiveness

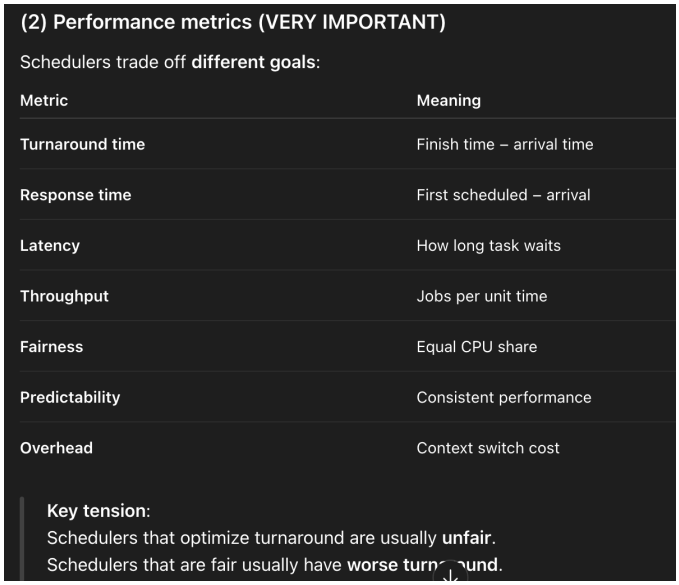
Every scheduler can be understood using **four ingredients**

(1) Workload assumptions

Early schedulers assume:

- Jobs are CPU-bound
- Runtime is known
- No I/O
- All jobs arrive at once

These assumptions are **deliberately unrealistic** — they help us reason.



KIV: Starvation, Shortest job First

(3) Scheduling decision

Given READY tasks, pick:

- **Which task**
- **For how long (quantum)**

Preemptive Scheduling (Modern OS schedulers)

- **Definition: The operating system can forcibly take the CPU from a running process and allocate it to another, often based on time slices or higher priority.**
- **When it occurs: Process time slice expires, a higher-priority process arrives, or the process moves to a waiting state.**
- **Pros: Better responsiveness, prevents CPU monopolization, ideal for interactive systems (Windows, Linux).**
- **Cons: Higher overhead (context switching), more complex to implement, potential for starvation of low-priority tasks.**
- **Examples: Round Robin, Shortest Remaining Time First (SRTF), Preemptive Priority.**
-

Non-Preemptive Scheduling

- **Definition: A process keeps the CPU until it finishes its CPU burst or voluntarily yields control (e.g., for I/O).**
- **When it occurs: Process finishes or enters a waiting state (I/O, termination).**
- **Pros: Simpler, less overhead (fewer context switches), deterministic.**
- **Cons: Can lead to long waits (convoy effect), less responsive, a long process blocks shorter ones.**
- **Examples: First-Come, First-Served (FCFS), Shortest Job First (SJF).**
-

Key Differences Summarized

- **Control: Preemptive allows interruption; Non-preemptive doesn't.**
- **Flexibility: Preemptive is flexible; Non-preemptive is rigid.**
- **Overhead: Preemptive has high overhead; Non-preemptive has low overhead.**
- **Responsiveness: Preemptive is better; Non-preemptive is worse.**

---

2. Classic Scheduling Policies

These are the **building blocks** — everything later is a response to their flaws.

FIFO / FCFS (First-In First-Out)

Rule: run tasks in arrival order

Properties

- Simple
- No preemption
- One queue

Fatal flaw: Convoy Effect

A long job blocks all short jobs behind it.

Good for:

- Simple batch systems

Bad for:

- Turnaround time
- Responsiveness
- Interactivity

Shortest Job First (SJF)

Rule: run job with **smallest total runtime** first

Key result

SJF is optimal for average turnaround time
(when runtimes are known and all arrive together)

Problems

- Runtime usually **unknown**

- Non-preemptive → late arrivals suffer

Shortest Time-to-Completion First (STCF)

Preemptive SJF

Rule

When a new job arrives:

- Preempt current task
- Run job with **least remaining time**

Pros

- Optimal turnaround time
- Handles late arrivals

Cons

- Terrible **response time**
- Interactive jobs suffer

---

3. Round Robin (RR)

Rule: Each task runs for a fixed **time quantum**, then rotates

Why RR exists

Interactive systems care about **response time**, not turnaround.

Trade-off

- Short quantum → responsive but high overhead
- Long quantum → efficient but sluggish UI

RR is:

- **Fair**
- **Responsive**
- **Terrible for turnaround time**

RR intentionally stretches jobs out.

---

4. Reality Check: I/O + Interactivity

Real programs:

- An **I/O-bound** task typically does a *short CPU burst* (e.g., compute next request, copy buffers, issue a syscall), then it **blocks** waiting for disk.
- While it's blocked, the **CPU would otherwise be idle** unless you schedule something else.

So if the scheduler quickly runs I/O-bound tasks and lets them issue their disk requests early, you get **overlap/pipelining**:

1. Run I/O-bound task briefly → it issues a disk request → it blocks.
2. While disk works, run some other ready task (often CPU-bound).

3. When disk finishes, the I/O-bound task becomes ready again and can quickly consume the result and issue the next I/O.

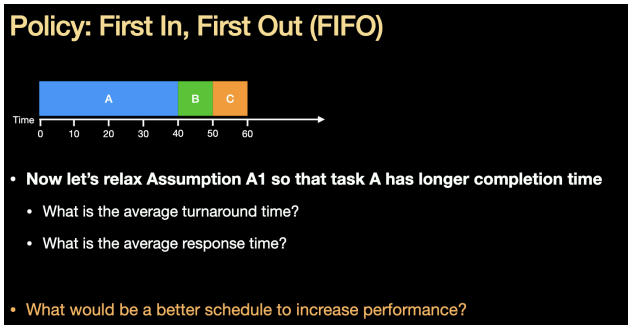
That increases:

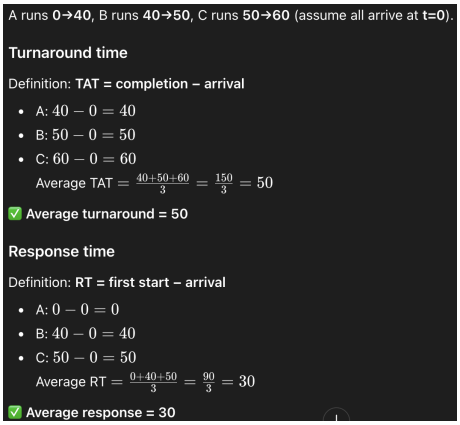
- **Disk utilization** (disk always has work queued instead of waiting for the next request to be issued),
- **CPU utilization** (CPU keeps running other tasks while I/O waits),
- and often **throughput** (more total work per unit time) and **responsiveness** (interactive/I/O tasks finish their "little bursts" quickly).

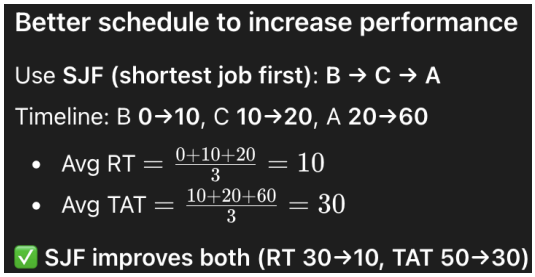
Schedulers must:

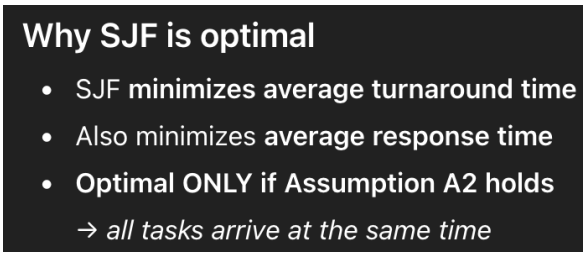
- Prefer I/O-bound jobs
- Avoid CPU idle time
- Handle wakeups intelligently

This motivates **feedback-based schedulers**.









OS NOTES

What happens if we relax Assumption A2?

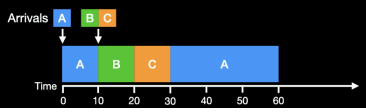
(tasks arrive at different times)

- A long job (A) may start **before** short jobs (B, C) arrive
- When B and C arrive, **SJF cannot preempt A**
- Short jobs are forced to wait
- **Optimality is lost**

⚠️ Result:

- SJF no longer guarantees minimal turnaround/response
- Can cause poor responsiveness

- **STCF is a preemptive variant of SJF**
  - Preemptive schedulers can take away resources from running tasks
- **When a new task arrives, policy updates the schedule**



Does STCF have good average response time?

Yes — usually very good average response time, because:

- When a short job arrives, STCF **immediately gives it the CPU** (preempts the long job),
- so short jobs start quickly → **low response times for short/interactive tasks**.

For the figure (response time = first start − arrival):

- A: starts at 0, arrives 0 → **0**
- B: starts at 10, arrives 10 → **0**
- C: starts at 20, arrives 20 → **0**

✅ Average response time here = $(0 + 0 + 0)/3 = 0$ (as good as it gets)

The tradeoff (important)

- STCF can cause **starvation**: a long job might keep getting delayed if many short jobs keep arriving.
- Also more **context-switch overhead** (because preemption happens).

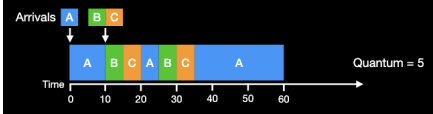
One-liner takeaway:

✅ **STCF is great for average response + turnaround (when times are known)**, but ⚠️ can starve long jobs without fairness controls (aging, priorities, RR, ↓ .).

OS NOTES

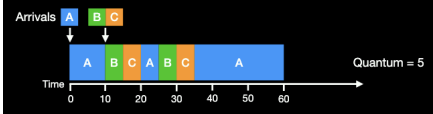
Policy: Round Robin (RR)

- **Runs each task for some fixed time slice (or scheduling quantum)**
  - Switches between tasks in order



- **What happens to average turnaround time?**

Choice of Scheduling Quantum



- **How do we choose the right quantum?**
  - What hardware state to tasks build up over time?
  - What is involved in switching between tasks?

Round Robin (RR) — what the slide is showing (Quantum = 5)

RR rule: run each ready task for **1 quantum** (5 time units), then move to back of queue (preempt if not done).

Arrivals in slide:

- **A arrives at t=0**
- **B and C arrive at t=10**
  (Burst times from earlier example: **A=40, B=10, C=10**)

Timeline in the picture (grouping consecutive A quanta)

- **0–10:** A (A was alone → gets 2 quanta back-to-back)
- **10–15:** B
- **15–20:** C
- **20–25:** A
- **25–30:** B (finishes)
- **30–35:** C (finishes)
- **35–60:** A (finishes)

What happens to average turnaround time?

Completion times:

- **A finishes at 60**
- **B finishes at 30**
- **C finishes at 35**

Turnaround = finish − arrival:

- $T_A = 60 - 0 = 60$
- $T_B = 30 - 10 = 20$
- $T_C = 35 - 10 = 25$

Average turnaround:

$$\frac{60 + 20 + 25}{3} = \frac{105}{3} = 35$$

✅ **Avg turnaround under RR here = 35**
(Usually **worse than SJF/STCF**, because short jobs get "chopped up" + wait between slices.)

Choice of scheduling quantum (why it matters)

- **Small quantum:** great **response time/fairness**, but **more context switches → overhead ↑ → turnaround ↑**
- **Large quantum:** RR starts looking like **FIFO** (better throughput, worse interactivity)

What hardware state builds up over time?

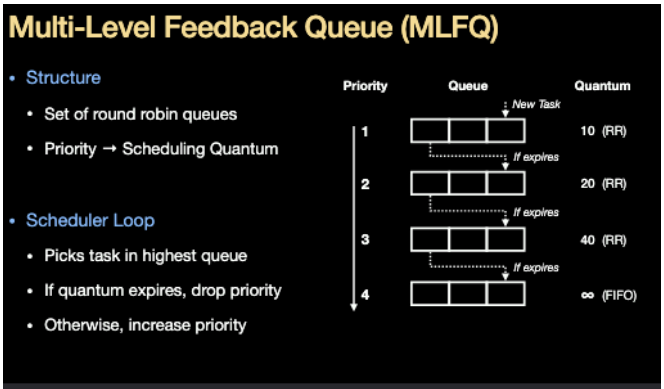
- **Caches (L1/L2/L3), TLB entries, branch predictor, pipeline warm-up**, working-set residency

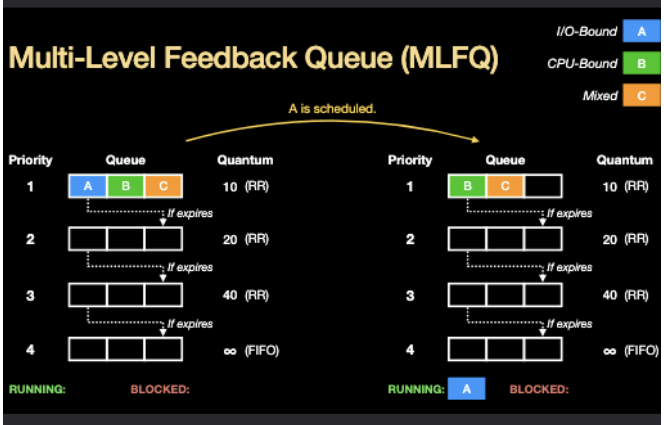
What's involved in switching tasks?

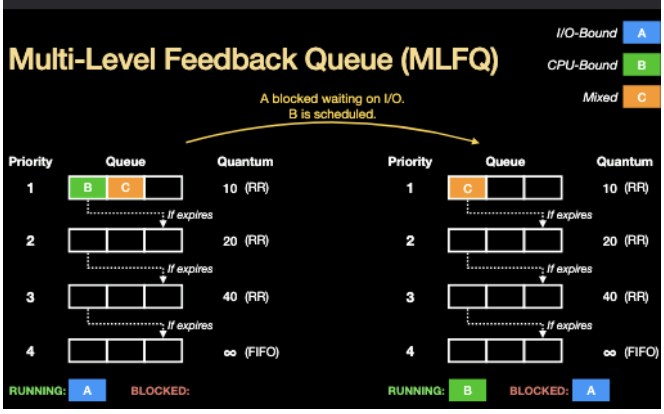
- Save/restore **PC, SP, registers**, kernel bookkeeping (PCB), switch address space/page table (may affect **TLB**), lose cache locality → **extra stalls**

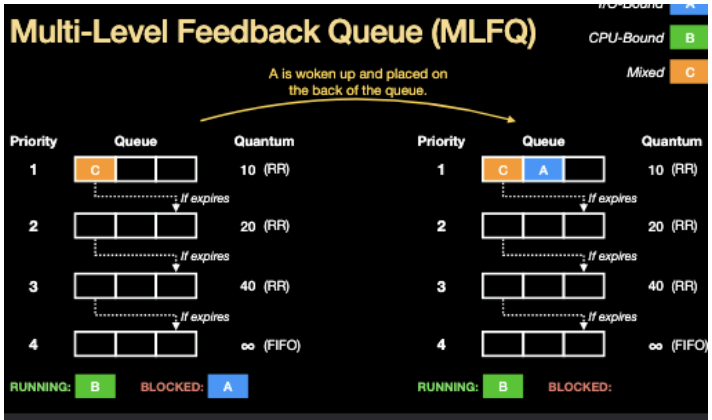
Focused on CPU resources, for other devices?

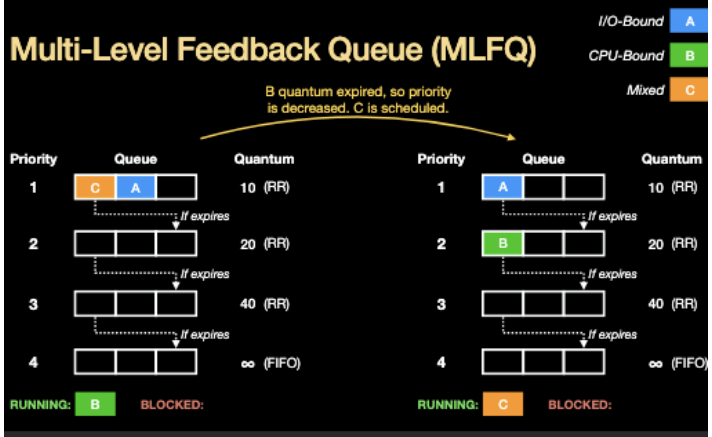
OS NOTES

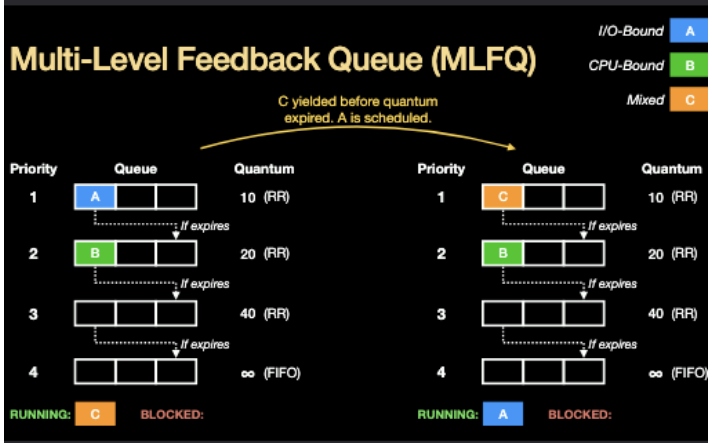

Multi-Level Feedback Queue (MLFQ)
Structure
Set of round robin queues
Priority → Scheduling Quantum
Scheduler Loop
Picks task in highest queue
If quantum expires, drop priority
Otherwise, increase priority
Priority
Queue
Quantum
New Task
1
10 (RR)
2
20 (RR)
3
40 (RR)
4
∞ (FIFO)
If expires

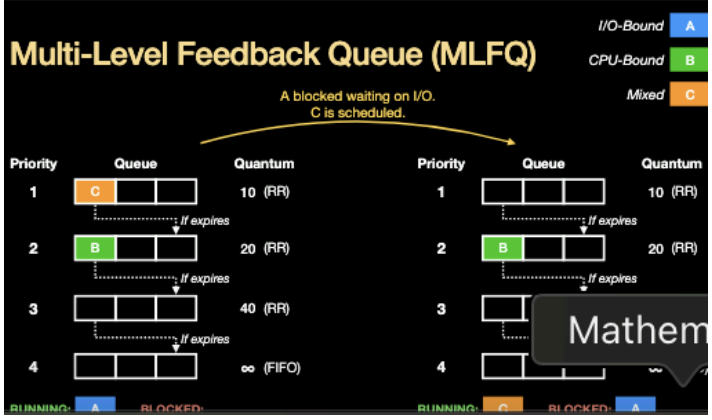

Multi-Level Feedback Queue (MLFQ)
I/O-Bound
CPU-Bound
Mixed
A is scheduled.
Priority
Queue
Quantum
RUNNING:
BLOCKED:


Multi-Level Feedback Queue (MLFQ)
I/O-Bound
CPU-Bound
Mixed
A blocked waiting on I/O.
B is scheduled.
Priority
Queue
Quantum
RUNNING:
BLOCKED:


Multi-Level Feedback Queue (MLFQ)
CPU-Bound
Mixed
A is woken up and placed on
the back of the queue.
Priority
Queue
Quantum
RUNNING:
BLOCKED:


Multi-Level Feedback Queue (MLFQ)
I/O-Bound
CPU-Bound
Mixed
B quantum expired, so priority
is decreased. C is scheduled.
Priority
Queue
Quantum
RUNNING:
BLOCKED:


Multi-Level Feedback Queue (MLFQ)
I/O-Bound
CPU-Bound
Mixed
C yielded before quantum
expired. A is scheduled.
Priority
Queue
Quantum
RUNNING:
BLOCKED:


Multi-Level Feedback Queue (MLFQ)
I/O-Bound
CPU-Bound
Mixed
A blocked waiting on I/O.
C is scheduled.
Priority
Queue
Quantum
Mathem

OS NOTES

Fundamental MLFQ Rules

Rule 1 — Priority dominates

If `Priority(A) > Priority(B)`, run A.

Rule 2 — Same priority → RR

Tasks at same level are round-robin scheduled.

Priority Adjustment Rules (Learning Behavior)

Rule 3 — New jobs start high

Assume job is interactive until proven otherwise.

Rule 4 — CPU usage reveals behavior

- **Uses full quantum** → CPU-bound → demote
- **Yields early (I/O)** → interactive → stay high

This is how MLFQ **learns job type**.

6. MLFQ Problems (Very Exam-Relevant)

(1) Starvation

Too many interactive jobs → CPU-bound jobs never run.

(2) Gaming the scheduler

A job can:

- Run almost full quantum
- Yield via fake I/O
- Stay high priority forever

(3) Phase changes

CPU-bound → interactive later

Scheduler must adapt.

Issue: Priority Inversion (what the slide means)

Setup (A, B share a resource)

- **A = high priority**
- **B = low priority**
- They share a lock/resource (ex: **pipe**, mutex, etc.)
- **A blocks waiting for B** (ex: A does `read()` on an empty pipe → A sleeps until someone writes / until B releases resource)

Where inversion happens (add C = medium priority)

1. **B (low)** acquires the resource first (holds the lock / is the one that must run to make progress).
2. **A (high)** tries to use it → **A blocks** ("waiting on B").
3. Scheduler now picks highest *ready* task. **C (medium)** is ready, so **C runs**.
4. **B can't run** (because C keeps preempting it), so **B can't release resource**.
5. So **A (highest priority) is stuck indirectly behind C** even though C is lower priority than A.

✅ That's the "inversion": **A > C > B** by priority, but effectively it becomes **C blocks A** (because C prevents B from running).

Solution: Priority Inheritance (what the diagram shows)

When **A is blocked on B**, the OS temporarily **raises B's priority up to A's level** (or at least above C):

- Now **B runs (even though it's normally low)**,
- B finishes the critical section / releases the resource,
- A wakes up and runs,
- Then B's priority drops back to normal.

Key line from slide:

If **P is waiting on Q**, then **Q inherits P's priority** (if higher than Q's).

OS NOTES

Summary

- **First-In, First-Out (FIFO)**
  - A1-A5 → Optimal for avg turnaround/response time
  - A2-A5 (~~A1: Uniform Length~~) → Can have poor avg turnaround/response time
- **Shortest Job First (SJF)**
  - A2-A5 → Optimal for avg turnaround/response time
  - A3-A5 (~~A2: Simultaneous Arrival~~) → Can have poor avg turnaround/response tim

Summary

- **Shortest Time-to-Completion First (STCF)**
  - Preemptive version (~~A3: Run to Completion~~) of **SJF** on task arrival
  - A4-A5 → Optimal for avg turnaround time, but not response time
- **Round Robin (RR)**
  - Preemptive based on scheduling quantum, not task arrival
  - A4→ Fair (and avoids starvation), good for avg response time

- **MLFQ**
  - Handles I/O (~~A4: CPU Bound Only~~) without oracle (~~A5: Known completion time~~)
  - Captures interactivity based on expiration of quantum
  - Leverages multiple queues that allow for spectrum of interactivity

---

MULTICORE SCHEDULING

The slide says: “Consider running a scheduler (e.g., MLFQ) on a multi-core processor… straightforwa
solution: maintain single data structure accessed by all cores… issues?” lecture_14_scheduling

What “single data structure” means

Imagine you have **one global READY queue** (or multiple MLFQ queues) shared by all CPU cores:

- Core 1, Core 2, Core 3 all look at the **same** queue(s) to pick the next runnable thread.

Why this becomes a problem

Because multiple cores try to:

- **Remove a thread** to run it
- **Insert threads** when they become READY (wakeup, fork, yield)
- **Move threads between priority levels** (MLFQ demotion/promotion)

So you need a **lock** around that shared structure.

The two big issues the slide hints at

1. **Lock contention**
   - Many cores compete for the same lock.
   - The scheduler can become the bottleneck: cores waste time waiting just to pick a thread.
2. **Cache contention / cache-line bouncing**
   - Even if the lock is “fast”, the *cache line* holding the lock and queue metadata bounces between cores. NEED LOCK

This produces major slowdowns because each core invalidates the others’ cached copy. So “straightforward” works conceptually but can scale horribly.

OS NOTES

Slide 3: Adapting MLFQ for Multiprocessors (Per-core READY queues): **What per-core READY queues are**

Instead of 1 global structure, we have:

- Core 1 has its own READY queue(s)
- Core 2 has its own READY queue(s) etc.

Now each core's scheduler usually only touches its own queue → far less locking.

Why "place tasks back on the same core" helps

That's **cache locality**:

- When a thread runs on Core 1, its working set (instructions/data) ends up in Core 1's caches (L1/L2).
- If you wake it up and run it again on Core 1, you reuse warm caches → faster.
- If you run it on a different core, caches are cold → more misses.

Core 1 might have tons of runnable threads, Core 2 might be idle (empty queue)So you fixed contention but introduced **load imbalance**.

Slide 4: Work Stealing

The slide says: "Cores without work can steal… per-core lock… if nothing to schedule, for each other core lock+inspect+move threads… issues?"

What "work stealing" is

If Core A is idle:

- It tries to take ("steal") runnable threads from Core B's queue.

Each core queue has its own lock.
Idle core loops over other cores:

1. lock that core's queue
2. check if there's a runnable thread to steal
3. maybe move one thread over
4. unlock

Why this can still be messy

1. **Stealing overhead**
- If you have many cores, scanning others can cost a lot.
2. **Lock thrashing**
- Idle cores may hammer locks on busy cores. a bunch of idle CPU cores keep repeatedly grabbing (or trying to grab) the locks that protect other cores' run queues, and that constant lock traffic slows everyone down, especially the busy cores.
3. **Cache locality loss**
- Stolen thread now runs on a different core (cold cache).
4. **Policy questions**
- Which thread do you steal? Highest priority? Lowest? Half the queue? One task?
- If it's MLFQ, do you steal from which priority level?

Linux Scheduler Evolution (Slides 6–25)

Slide 6: Timeline

Linux went through: RR → O(N) → O(1) → CFS → EEVDF

This is basically the story:

- Early: simple but not scalable/fair enough
- Middle: optimize runtime cost and priorities
- Later: aim for *fairness* + *latency guarantees* under real workloads

---

Slide 7: Pre-v2.4 Round Robin

Slide says: “single circular queue + lock… fixed time quantum… similar to xv6… pros/cons?”

How it works

- Think one ring of tasks: A → B → C → A → …
- Each gets a time slice (quantum).
- After quantum, move to next.

Pros

- Simple
- Reasonable response time

Cons

- No good notion of priority/fairness beyond “everyone equal”
- Doesn’t scale to many tasks / many cores (global lock)
- Turnaround time for short jobs can be bad (they get sliced

Slide 8: v2.4 O(N) Scheduler

Slide says: “each epoch all tasks execute up to slice; leftover half carried; scheduler scans all tasks to find best ‘goodness’; based on niceness + remaining slice”

Key concepts

- **Epoch**: a window where each task gets some slice budget.
- After running, leftover time slice partially carries forward.

“Goodness” means

Scheduler scores tasks, then picks the best score.
But to pick it, it may need to check **every runnable task**.

Why O(N) is bad

If you have N tasks, choosing next can cost O(N) time → slow for large N.

Slide 9–10: v2.6 O(1) Scheduler (Active/Expired + Bitmaps)

Slide says: “two sets of per-core queues; scheduled task goes active→expired; when active empty swap pointers; priorities RT [0..99], normal [100..139] …”

Then: “quantum depends on priority; load balancing every 200ms; work stealing if empty.”

Why it’s called O(1)

Picking next task is constant-time because:

- Each priority level has a queue.
- A **bitmap** tells which priorities are non-empty.
- “Find first set bit” → immediately find highest priority runnable queue.

Active vs Expired

- **Active queues**: tasks that still have time slice left in this round.
- When a task uses up its slice, it goes to **expired**.
- When all active empty → swap active and expired pointers (cheap).

Quantum depends on priority

Higher priority generally gets larger or more favorable slices (the slide gives formulas and example values).

Pros

- Very fast to pick next task (great scalability vs O(N))
- Supports priorities

Cons (why Linux moved on)

- Fairness and “interactive responsiveness” relied on heuristics
- Complex tuning; behavior could feel inconsistent

Slide 11–15: CFS (Completely Fair Scheduler)

Scheduling classes (why)

Linux doesn’t treat all tasks the same:

- Some are real-time
- Some have deadlines
- Normal user tasks
  So Linux uses “classes” stacked by strict priority.

What CFS is trying to achieve

Fairness in a very specific sense: If N equal tasks are runnable on one core, each should get ~1/N of CPU time. But tasks have different nice values → different weights → different shares.

vruntime (the core trick)

Instead of tracking “real time run”, CFS tracks **virtual runtime**:

- If you run more than your fair share, your vruntime increases faster.
- Scheduler always chooses the task with **smallest vruntime** → the one “most behind” in fair share.

Why a red-black tree

You need:

- Insert runnable task, Remove runnable task, Find min vruntime task quickly

A balanced tree gives:

- O(log N) insert/remove
- O(1) to get leftmost min node (conceptually)

How nice affects vruntime (important formula meaning)

Slide gives: A.vruntime += t * (weight[20] / weight[20 + priority])

Interpretation:

- If a task has **higher weight** (higher priority), denominator bigger → multiplier smaller
- So for same real runtime t, its vruntime grows **slower**
- That means it stays “more eligible” longer → gets more CPU share

Wakeup adjustment (preventing “sleep advantage”)

If a task sleeps for a long time:

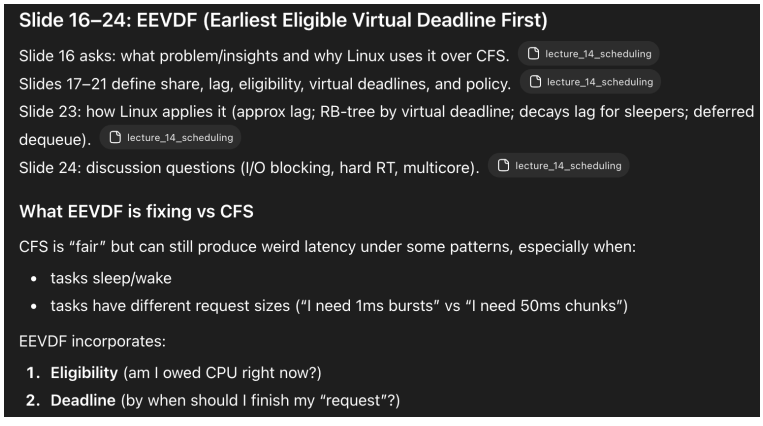
- It hasn’t accumulated vruntime
- If you wake it with tiny vruntime, it would always be chosen and could monopolize.

So Linux adjusts: When a task becomes ready, set its vruntime near current minimum.

How long does CFS run a task?

Not fixed RR quantum; it uses target latency ideas:

- Want each runnable task to run at least once within a “latency” window.
- So slice ≈ sched_latency / #tasks but never below a minimum granularity.



OS NOTES

Resource share math (what fi(t) means)

Slide says:

$$f_i(t) = \frac{w_i}{\sum_{j \in A(t)} w_j}$$

This is just:

- If total runnable weight is W, and task i weight is wi,
- then i's fair fraction of CPU is wi/W. lecture_14_scheduling

Then it defines the *ideal* service between t0 and t1:

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau)\, d\tau$$

Meaning:

- "If the world were perfectly fair continuously, task i should have gotten this much CPU time in that interval." lecture_14_scheduling

Lag (what it measures)

Slide defines:

$$lag_i(t) = S_i(t_0^i, t) - s_i(t_0^i, t)$$

where s_i is the *actual* service time. lecture_14_scheduling

Interpretation:

- lag > 0 means "I am behind; I'm owed CPU"
- lag < 0 means "I've already gotten more than my share recently"

Eligibility rule

Slide says: eligible if lag is positive. lecture_14_scheduling

So EEVDF won't even consider tasks that are "ahead".

Virtual time V(t)

The slide's V(t) is a way to normalize fairness with weights. lecture_14_scheduling

Practical meaning:

- It turns "real CPU time under varying total runn[illegible] weight" into a consistent fairness clock.

Virtual deadline

Slide gives:

$$V(d) = V(e) + \frac{r}{w_i}$$

Where r is request length (in real time units) scaled by weight. lecture_14_scheduling

Interpretation:

- Each task says: "I have a request of size r"
- If I deserve wi share, then in virtual time, finishing that request corresponds to adding r/wi.
- That target is a "deadline" in virtual time.

EEVDF policy (the one-liner)

> Allocate quantum to the eligible client with the **earliest virtual deadline** lecture_14_scheduling

So:

1. filter by eligibility (lag > 0)
2. among those, choose smallest deadline

Linux makes approximations:

- Lag approximated by vruntime vs average vruntime
- Eligible time treated as vruntime
- Keep RB-tree sorted by virtual deadline
- Decay lag for sleepers (so long-sleeping tasks don't come back with huge positive lag and instantly dominate)

FILE I/O: A) What apps want (goals)

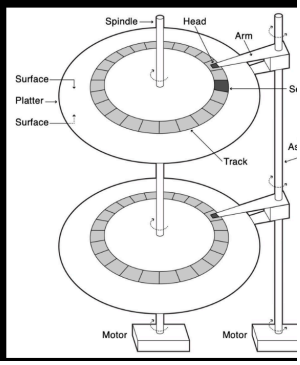
• Persistence: Provide long-term, continued storage of application data
• Hardware may be susceptible to crashes or other failure conditions
• Need recovery mechanisms to provide Persistence
    • Example: System loses power during an update
    • Example: Some of data on the hardware disk becomes corrupted

• Naming: Higher-level means of "addressing" data
• Support mapping of names to data (files)
• Support hierarchical organization for ease-of-use (directories)
    • Hardware disks provide flat, block-level storage and access
        • Need to resolve byte-level access to block-level access
        • Need to map the structure (files, directories) onto block-level storage

• Performance: High-throughput, low-latency access to stored data
• Provide performance as close to hardware limit as possible
• Disks have varying performance properties (e.g., slower than memory)
• Need to support for caching data in memory
• Need to consider placement of data on disks
• Need to consider how to schedule accesses to disks

Hard Disk Drive (HDD) - Implementation

- Disks have one or more Platters
  - Platters have two Surfaces (above/below)
- Each Surface has multiple Tracks
- Each Track has multiple Sectors
  - Each Sector holds 4 KB (512 B in the past)
  - Outer Tracks hold more Sectors
- The Arm/Head is used to access data
  - Sense magnetic pattern (read) or induce a change in the pattern (write)



Seek time: move head to the right track
Rotation time: wait for the right sector to rotate under head
Transfer time: actually read bytes
Random I/O is slow because seek + rotation dominates. Sequential is faster because you pay seek/rotation less often.

OS NOTES

Size	
Platters/Heads	2/4
Capacity	320 GB
Performance	
Spindle speed	7200 RPM
Average seek time read/write	10.5 ms/ 12.0 ms
Maximum seek time	19 ms
Track-to-track seek time	1 ms
Transfer rate (surface to buffer)	54–128 MB/s
Transfer rate (buffer to host)	375 MB/s
Buffer memory	16 MB
Power	
Typical	16.35 W
Idle	11.68 W

Specification for Toshiba Disk

Transfer (Inner) = 512B / 54 MB/s = 9 us
Transfer (Outer) = 512B / 128 MB/s = 4 us

Disk

- Spindle speed: **7200 RPM**
- Avg seek time: **10.5 ms**
- Sector size: **512 B**
- Transfer rate: **54–128 MB/s**

Step 1: Rotational latency (must compute)

7200 RPM

$$= \frac{7200}{60} = 120 \text{ rev/s}$$

Time per full rotation:

$$T_{\text{rot, full}} = \frac{1}{120} = 8.33 \text{ ms}$$

Average rotational latency (half rotation):

$$T_{\text{rot}} = \frac{8.33}{2} = 4.17 \text{ ms}$$

Step 2: Transfer time per sector

Worst case (inner track, slowest):

$$T_{\text{transfer}} = \frac{512 \text{ B}}{54 \text{ MB/s}} = \frac{512}{54 \times 10^6} \approx 9 \times 10^{-6} \text{ s} = 0.009 \text{ ms}$$

(Outer track would be ~0.004 ms — same conclusion.)

Case 1: 100 random sector reads

Each read requires:

- **Seek**
- **Rotation**
- **Transfer**

Time per random read

$$T_{\text{random,1}} = 10.5 + 4.17 + 0.009 = 14.68 \text{ ms}$$

Time for 100 random reads

$$T_{\text{random,100}} = 100 \times 14.68 = 1468 \text{ ms} \approx \boxed{1.5 \text{ s}}$$

Case 2: 100 random sequential sector reads

Meaning:

- 1 random seek + rotation
- then 100 contiguous sectors (no more seeks)

Initial positioning cost (once)

$$T_{\text{position}} = 10.5 + 4.17 = 14.67 \text{ ms}$$

Sequential transfer cost

$$T_{\text{transfer,100}} = 100 \times 0.009 = 0.9 \text{ ms}$$

Total time

$$T_{\text{seq,100}} = 14.67 + 0.9 = 15.57 \text{ ms} \approx \boxed{15.6 \text{ ms}}$$

OS NOTES

Disk scheduling (HDD)

Because seeks dominate, the OS/driver may reorder requests:

- **SSTF (Shortest Seek Time First)**: good avg seek, but can **starve** far-away requestshat is the policy?

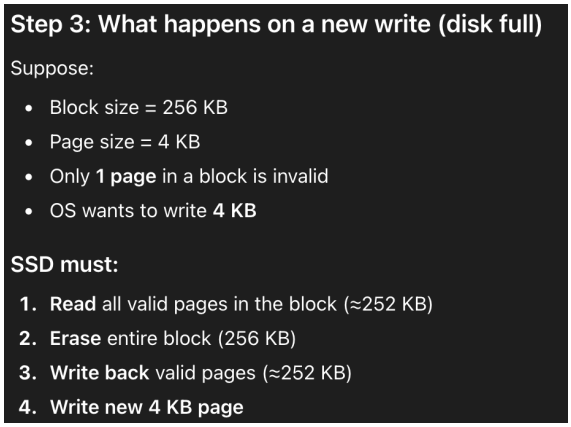
  • Sort the queue of I/O requests by track • Pick request closest to current arm/head position

SCAN/Elevator: Move arm in one direction then reverse (rinse and repeat). All requests are satisfied along the way, reduces starvation risk

B) SSD internals (why writes are weird)

WRITE AMPLIFICATION: actual amount of information physically written to the storage media is a multiple of the logical amount intended to be written. So a small write forces:

- read old block
- merge new data
- erase block
- write block back



Every erase requires copying **valid data**

SSD slides emphasize **asymmetry**:

- Changing a bit from 0 to 1 is easy, but changing a bit from 1 to 0 is complicated!
- • Cells are grouped into larger “erase blocks” (e.g., 1 MB to 8 MB)
- • Erasure is performed all or nothing for each block
- • Additionally, erasing blocks causes wear (limited lifetime per block)

So: even though SSDs are “fast”, their *internal constraints* motivate **log-like designs**

OS NOTES

Key fact (the whole game-changer)

SSDs have no moving parts

⇒ **Seek time = 0**

⇒ **Rotation time = 0**

So the latency model **collapses**.

SSD Access-Time Model

HDD (for comparison)

$$T_{\text{HDD}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}$$

SSD (reads)

$$T_{\text{SSD, read}} = T_{\text{controller}} + T_{\text{flash-read}} + T_{\text{transfer}}$$

SSD (writes)

$$T_{\text{SSD, write}} = T_{\text{erase}} + T_{\text{program}} + T_{\text{transfer}}$$

Random I/O is where SSDs destroy HDDs

Sequential Reads/Writes: HDD vs SSD

HDD sequential

- Seek + rotation once
- Then streaming at **100+ MB/s**

SSD sequential

- No positioning cost
- Limited only by interface (SATA/NVMe)

Typical:

- SATA SSD: ~500 MB/s
- NVMe SSD: 3–7 GB/s

Result

Sequential improvement:

$$\sim 5\text{–}20\times$$

➡️ **Sequential gains are real but not as dramatic as random gains**

What changes from HDD → SSD for random I/O?

- Seek = 0
- Rotation = 0
- Latency drops from **~15 ms → ~100 µs**

What about sequential I/O?

- HDD amortizes seek/rotation
- SSD still faster, but improvement is **bandwidth-limited**, not latency-limited

OS NOTES

DISK DRIVER

• Provides a generic block interface over a specific storage device
• Typically, you will have:
• An array of descriptors for specifying operations (e.g., read a block)
• Queues for descriptors that are: New/In-Progress or Finished
• Interrupt handler for notification of newly finished descriptors
• Format of descriptors depends on the specific storage device interface
• For xv6, this is the VIRTIO Disk interface

1) Block / Buffer Cache

Why is it useful?

A disk (HDD/SSD) is **orders of magnitude slower** than RAM. If every `read()` had to hit the disk, programs would crawl.

A **buffer cache** (aka block cache / page cache depending on OS) keeps **recently used disk blocks in memory**, so:

- **Reads become fast** if the block is already cached (**cache hit**).
- **Writes can be delayed/batched**: write into memory first, flush later → fewer disk operations, better throughput.
- It helps with both:
  - **Temporal locality**: same blocks are used again soon (metadata blocks, directories, inodes).
  - **Spatial locality**: nearby blocks are used soon (sequential file access).

How does it work (mechanically)?

Think in **blocks** (e.g., 1KB in xv6, 4KB common elsewhere). The cache stores objects like:

- `(dev, blockno) → buffer`
- `buffer` contains:
  - data bytes for that disk block
  - metadata: valid?, dirty?, refcount?, lock?

Read path

1. Process calls `read(fd, user_buf, n)`.
2. Kernel translates `(fd, file offset)` → which **disk block number(s)** are needed.
3. For each needed block:
   - Check cache for `(dev, blockno)`.
   - **Hit**: copy bytes from cached block into user buffer.
   - **Miss**: allocate a buffer, issue disk I/O, sleep until completion, mark valid, then copy.

Write path

1. Process calls `write(fd, user_buf, n)`.
2. Kernel finds target disk blocks.
3. Bring blocks into cache (if not already there).
4. Copy user bytes into cached buffers.
5. Mark buffers **dirty**.
6. Actual disk write happens later:
   - periodically, or
   - when the FS forces it (e.g., journaling commit), or
   - when user calls `fsync()`.

Key concept: *delayed write* vs *durability*

- Delayed write improves performance but risks data loss on crash.

- `fsync(fd)` is the user-visible “I need this on disk now” boundary.

Replacement (when cache is full)

Cache needs an eviction policy:

- LRU-ish is common (evict least-recently-used block).
- But dirty blocks cost more to evict because you must flush first.

2) File System Abstraction

What is an inode? file? directory? path? link?

File: A **file** is logically:

- a **sequence of bytes** (0..N-1)
- with metadata: size, permissions, timestamps, etc.

The OS must map that logical byte sequence onto physical disk blocks.

Inode: An **inode** (“index node”) is the on-disk (and cached-in-memory) structure that stores:

- **metadata** (owner, mode/permissions, timestamps, size)
- **how to find the file’s data blocks**
  - direct/indirect pointers or extents
- **link count** (how many directory entries point to it)

Important: **names are NOT stored in the inode** in classic Unix design. Names live in directories.

Directory: A **directory** is a special file whose data is a table of:

- `name → inode number`

So “folders” are not magic: they are mappings that let us find inodes.

Path: A **path** is a string like `/usr/bin/ls`.
To resolve a path:

1. start at root inode `/` (or CWD for relative paths)
2. look up each component name in the directory
3. follow to next inode, repeat until final inode

Links: Two main kinds:

Hard link

- A directory entry that points directly to an inode number.
- Multiple names can refer to the same inode/data.
- Deleting one name (`unlink`) just removes that directory entry.
- File data is reclaimed only when **link count reaches 0** AND no process still has it open.

Symbolic link (symlink / soft link)

- A small special file whose contents is a **path string** to the target.
- If target moves/deletes, symlink can break (dangling symlink).
- Symlink can cross file systems; hard links usually cannot.

UNIX API (open, read, write, lseek, …)

`open(path, flags, mode)`

- Returns a **file descriptor (fd)**: a small integer index in the process table.
- Kernel creates/uses an **open file description** (kernel object) storing:
  - pointer to inode
  - current offset
  - access mode
  - refcount (shared across dup/fork)

`read(fd, buf, n)`

- Reads up to `n` bytes from file at **current offset**
- Copies into user memory `buf`
- Advances offset by number of bytes read
- Returns 0 on EOF

`write(fd, buf, n)`

- Writes bytes starting at current offset
- Advances offset
- May extend file size

`lseek(fd, offset, whence)`

- Changes the file's current offset
- `SEEK_SET`, `SEEK_CUR`, `SEEK_END`

`close(fd)`

- Drops the process's reference
- Kernel frees open-file state when refcount hits 0

`dup/dup2`

- Makes a new fd pointing to the same open-file description
- They SHARE the same file offset

`unlink(path)`

- Removes a directory entry
- Decrements inode link count
- Actual data freed only when link count is 0 AND no open refs exist

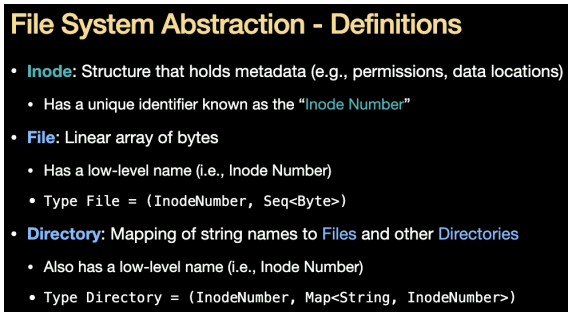
`fsync(fd)`

- Force dirty data + metadata (depending on FS) to stable storage
- The "durability barrier" that defeats delayed-write for that file

---

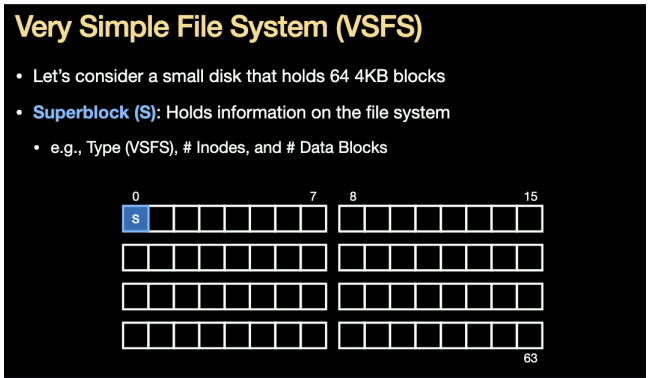
3) File System Internals

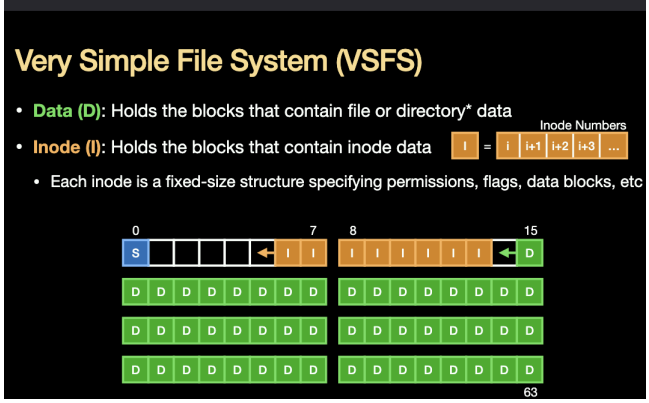
OS NOTES



- Path (or Pathname): String that identifies a File or Directory
- • Absolute Path: Fully specified starting from the root
- • Relative Path: Specified from a given current directory
- • "/" is used as the separator
- • ".." refers to parent directory, "." refers to current directory

• Files: Need to maintain metadata and sequence of blocks
• Directories: Need to maintain map of names to files (or other directories)
• Links: Need to maintain soft and hard links
• Free Space: Need to maintain set of free/available blocks





Pros

- Simple, fast enough for small workloads
- Good random access when using pointer indexing

Cons

- Limited maximum file size if pointer structure is small
- Fragmentation can happen depending on allocator
- Updating metadata can require many writes (reliability issue)

3.2 How do file systems store data for an inode?

VSFS-style (Very Simple FS)

- Inode has some number of direct pointers + maybe one indirect
- Free space via bitmaps
- Very clear/teachable layout: superblock, inode bitmap, block bitmap, inode table, data blocks

The inode stores an **array of block pointers**, plus pointers to *indirect blocks*.

A) Direct pointers

Inode contains `D` direct pointers:

- `inode.addrs[0..D-1]` point straight to data blocks

So file block `i` (if `i < D`) maps directly:

- `data_block = inode.addrs[i]`

Fast and simple for small/medium files.

B) Single indirect pointer

When file grows beyond direct pointers:

- inode has one pointer to an **indirect block**
- that indirect block is just an array of data-block numbers

So for file block `i` in the indirect range:

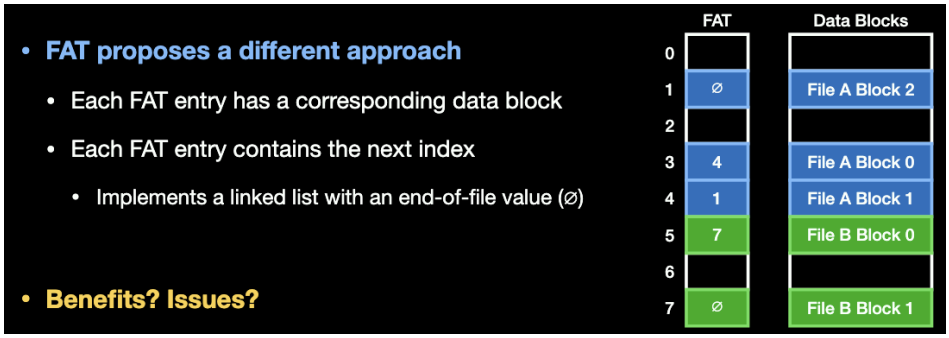
- index = `i - D`
- read indirect block
- `data_block = indirect[index]`

Pros

- Simple, fast enough for small workloads
- Good random access when using pointer indexing

Cons

- Limited maximum file size if pointer structure is small
- Fragmentation can happen depending on allocator
- Updating metadata can require many writes (reliability issue)



Pros

- Simple
- Appending is easy (update last pointer)

Cons

- Random access is bad: to reach block k you may traverse k pointers
- The FAT itself can be large and hot (cache pressure)
- Fragmentation hurts sequential performance

Workload fit

- Good for small/simple devices, removable media
- Not great for huge random-access files

FFS / “Unix inode with indirect blocks” (xv6-like)

Inode contains:

- **direct pointers** to data blocks (fast for small files)
- **single indirect** pointer → block of pointers
- sometimes double/triple indirect for very large files (classic Unix)

Pros

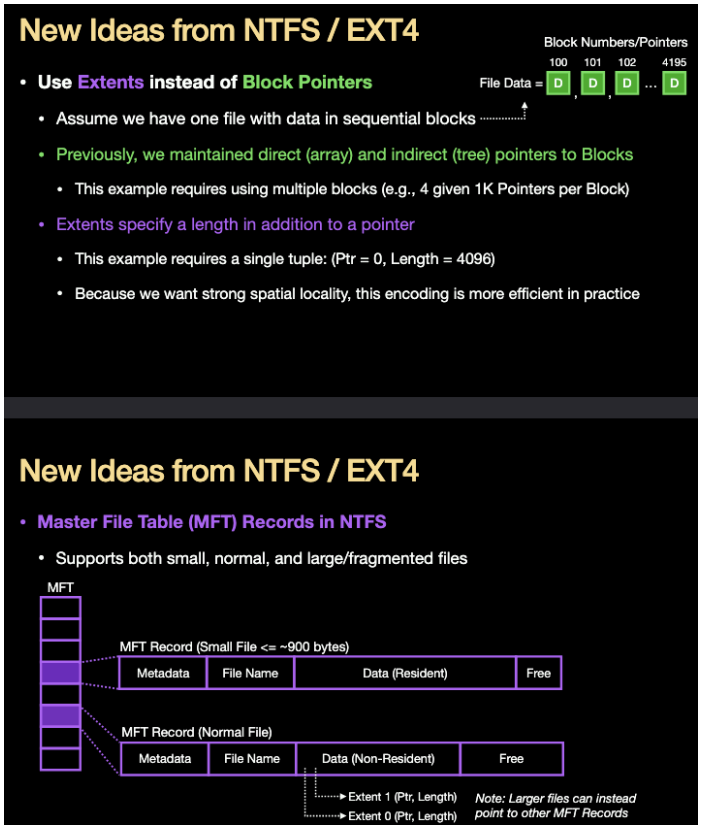
- Good random access: can compute/index to the block via pointer tree
- Efficient for small files (direct pointers avoid extra indirection)
- Scales to large files with indirect levels

Cons

- Indirect blocks add overhead for very large files
- Still can fragment without careful allocation policies

Workload fit

- General purpose; good mix of sequential and random



- **3.3 How NTFS/EXT4 extend designs**

Extents (EXT4/NTFS)

Instead of listing every block pointer, store runs:

- `(start_block, length)` meaning “a contiguous range”

Why it helps

- A large contiguous file might be represented by a few extents, not thousands of pointers
- Great for sequential I/O, less metadata, faster mapping

Tradeoffs

- If file is fragmented, you need more extents
- Need extent trees (metadata complexity)

Resident vs Non-Resident data (NTFS concept)

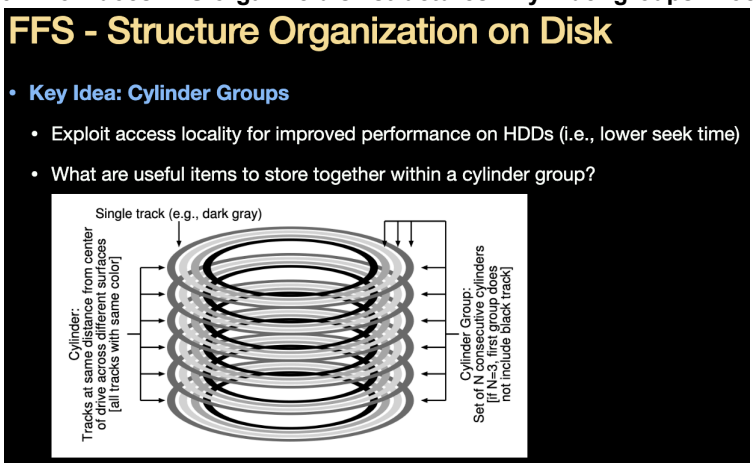
- **Resident**: very small file data stored inside metadata record (like “inode contains content”)
  - saves a separate data block read → small files become much faster
- **Non-resident**: file data stored in separate blocks/extents as usual

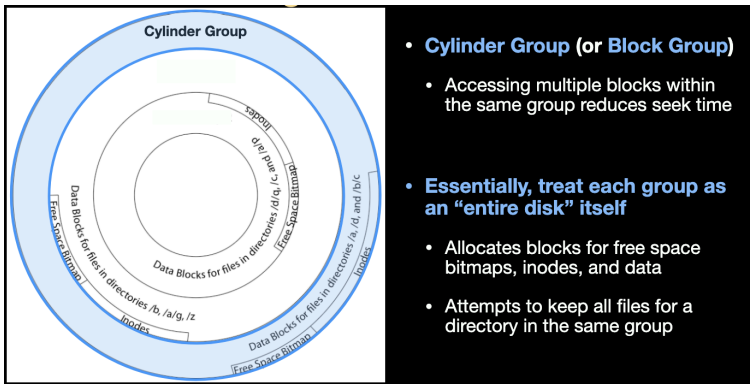
Why it matters

- Real systems have tons of tiny files (configs, logs, metadata)

- Resident storage is a big win for those

3.4 How does FFS organize disk structures? Cylinder groups + locality





Problem: seeks kill HDD performance

If inode is far from its data blocks, reading a file causes seeks:

- read inode block
- seek to data blocks
- seek to directory blocks, etc.

FFS idea: cylinder groups (block groups)

Partition disk into groups; each group contains:

- some inodes
- some data blocks
- bitmaps/metadata for that group

Then:

- put a file's inode + its data near each other
- put files from same directory near each other

Result

- Better locality → fewer seeks → faster on HDD

- Still relevant conceptually (even though SSD reduces seek cost)

---

4) Logging / Reliability

4.1 Hardware guarantees (what you can/can't assume)

Typical assumptions taught:

- Disk exposes **block reads/writes** (sector/block)
- A single sector write is often treated as "atomic" in theory, but in reality you must be careful:
    - Power loss can still create partial/garbled writes on some hardware without protection

• Writes at the sector granularity are atomic, Recall: For some disks, this may differ from the block size (e.g., 512 B vs 4 KB)

- **Disks may reorder pending writes**
- **• For instance, if multiple write operations are queued up at the same time**
- **• However, disks provide completion notifications which OS can wait on**

So from the OS perspective:

- You cannot assume "writes hit disk immediately"
- You cannot assume "writes reach disk in the order I issued them"
- Unless you use flush/barriers (`fsync`, cache flush, journal commit protocol)

4.2 Crash faults

Crash consistency

A filesystem is **crash consistent** if, after a crash and reboot, the on-disk state is:

- either the state before an operation,
- or the state after it,
- but not some impossible half-and-half that violates invariants.

Transaction

A **transaction** groups multiple low-level writes into one atomic unit:

- "all these updates happen" OR "none of them happen"

FS operations like "create a file" are inherently multi-write:

- allocate inode
- allocate data block(s)
- add directory entry

- update bitmaps
- update inode metadata

Crash mid-way can leave:

- allocated blocks not referenced (leaks)
- directory entry pointing to uninitialized inode (dangling)
- inode referencing blocks not marked allocated (double allocation)
- link counts wrong, etc.

4.3 Solutions

A) Careful ordering + fsck

Careful ordering
Choose write order to reduce dangerous states.
Example rule of thumb:

- write “data” before writing metadata that points to it,
- so metadata never points to garbage.

But ordering alone can’t fix everything, because many invariants involve multiple structures.

fsck (file system check)
On reboot, fsck scans whole disk to repair inconsistencies:

- rebuild free space info from inode pointers
- fix link counts
- find unreferenced blocks, put orphaned files into `lost+found`

Pros

- Works for simple file systems
- No extra logging overhead during normal operation

Cons

- Slow on large disks (full scan)
- Can only restore consistency, not necessarily user-intended latest data
- Big downtime after crash

B) Logging (Journaling)

Idea
Before modifying “home locations” (inodes, bitmaps, directories), write a description of the updates to a **log** on disk.

Two common styles:

Redo logging (common in journaling FS)

1. Write intended updates to journal
2. Write commit record
3. Apply updates to home locations
   Recovery:

- if commit exists → redo/apply again (idempotent)
- if no commit → ignore

Undo logging (less common for FS journaling)

- log old values first, then write home locations
- on crash, undo incomplete transactions

Most modern journaled file systems use redo-style metadata journaling.

Modes

- Metadata-only journaling: data blocks might be written outside journal (faster)
- Data journaling: data + metadata journaled (safer, slower)

Pros

- Fast recovery (replay small log, not scan whole disk)
- Strong crash consistency for journaled operations

Cons

- Extra writes (write amplification)
- Complexity (must handle ordering + commit protocol)

---

5) Fail-Stop Faults + RAID

What is a fail-stop fault?

A component (disk) **stops working** completely (dies, disappears, returns errors), rather than returning corrupted data silently.

So crash-consistency (power loss) is different from disk failure:

- Journaling fixes “crash during update”
- RAID addresses “a disk is gone”

RAID abstraction

OS NOTES

RAID makes **multiple physical disks look like one logical disk** with:

- improved performance and/or
- improved reliability via redundancy

Common RAID levels (what they do + performance)

RAID 0 (striping)

- Data split across disks for parallelism
- **Performance**: great (reads/writes spread out)
- **Reliability**: terrible (any disk failure loses array)
- **Capacity**: sum of disks

RAID 1 (mirroring)

- Each block stored on two disks
- **Reads**: can be faster (read from either)
- **Writes**: must write both (slower than RAID0)
- **Reliability**: can survive 1 disk failure (per mirror set)
- **Capacity**: half

RAID 5 (striping + distributed parity)

- Blocks striped; parity rotated across disks
- Survives **1 disk failure**
- **Reads**: good
- **Small writes**: expensive (read-modify-write parity)
    - must read old data + old parity, compute new parity, write both

RAID 10 (striped mirrors)

- Mirror pairs + stripe across pairs
- **Performance**: excellent
- **Reliability**: good (can survive multiple failures if not in same mirror)
- **Capacity**: half

Exam-style punchline

- RAID0 = performance only
- RAID1 = simplest redundancy
- RAID5 = redundancy with better capacity efficiency than RAID1, but parity write penalty
- RAID10 = best mix of performance + reliability (but capacity cost)

Reliability Option #2: Logging (Journaling)

Core idea (what problem we are solving)

Problem:
A single filesystem operation (e.g., create file, rename, write) requires **many disk writes**:

- inode updates
- bitmap updates
- directory entry updates
- Data block writes

A crash in the middle leaves the filesystem in an **impossible state**.

Solution:
👉 **Do not update filesystem structures directly.**
👉 **First write intentions to a log.**
👉 **Only apply changes after a commit marker.**

This turns a complex multi-write operation into a **transaction**.

What is the log?

- The **log is an append-only file** on disk
- It stores:
    - `TxBegin`
    - individual intended operations
    - `TxEnd` (commit record)

The log is the source of truth for recovery.

Disk structures may be half-updated — the log tells us what *should* have happened.

System State View (VERY IMPORTANT)

At any moment, the system state includes:

- Log contents (what transactions exist, committed or not)
- On-disk filesystem structures (may be partially updated)
- Buffer cache (dirty blocks not yet flushed)
- Transaction state (in progress / committed)

Your job is to track **how these change over time**.

Logging: How is it performed?

The Three Phases (this is CRITICAL)

Phase 1: Write transaction intent to the log

Input: Filesystem operation begins (e.g., `create("x")`)

State changes:

- Append `TxBegin` to log
- Append all intended operations:
    - “allocate inode #42”
    - “write directory entry (x → 42)”
    - “set inode metadata”

- NOTHING on the real filesystem has changed yet

📌 At this point:

- Log knows what we *want* to do
- Disk filesystem still unchanged

Phase 2: Write commit record (`TxEnd`)

Input: All intended operations are safely in the log

State change:

- Append `TxEnd` to the log

📌 This is the **atomic decision point**.

> Once `TxEnd` is on disk, the transaction is considered **committed**,
> even if real filesystem updates haven't happened yet.

Phase 3: Apply operations to disk ("home locations")

Input: Transaction is committed

State changes:

- Write inode blocks
- Write directory blocks
- Write bitmap blocks
- These writes can be:
  - delayed
  - reordered
  - interrupted by crashes

📌 **Correctness no longer depends on these writes finishing before a crash.**

Journaling phases & synchronization: what must be ordered and why

Key rule (the whole reason journaling works)

Recovery uses the log like this:

- **If no `TxEnd`** → pretend transaction never happened (ignore)
- **If `TxEnd` exists** → transaction *must happen* (redo from log)

So the **commit record (`TxEnd`) is the "truth line."** That creates two required implications:

1. `TxEnd` on disk ⇒ all log records for that tx are on disk
2. `TxEnd` on disk ⇒ it's OK if the "real FS" update didn't finish (redo will fix)

Everything about synchronization is just enforcing those two implications.

Phase 1 ordering: "log records must hit disk before commit"

What Phase 1 writes

`TxBegin` + operation records (the "redo info")

Why you must synchronize

Disks + caches can **reorder/delay** writes. Without a flush/barrier, you can get:

- `TxEnd` reaches disk
- but op records are still only in RAM (not durable)
- crash happens

- recovery sees `TxEnd` (committed!) but **cannot find complete ops to redo**

That's fatal because the commit says "this happened," but the log doesn't contain the data needed to make it happen.

✅ Therefore the OS must do:

1. write all Phase 1 log blocks
2. **flush them to stable storage** (wait for completion)
3. only then write `TxEnd`

In one line:
You can't allow commit to be durable unless the redo information is durable.

Phase 2 ordering: "commit must hit disk before treating FS updates as permanent"

What Phase 2 writes

`TxEnd` (commit record)

Why you must synchronize

Suppose you start writing the real filesystem updates (Phase 3) *before* `TxEnd` is durable, and then crash:

Bad scenario:

1. You apply some home-location updates (directory entry, bitmap, inode…)
2. But `TxEnd` never made it to disk
3. Crash
4. Recovery scans log: sees no `TxEnd` ⇒ treats tx as **uncommitted** ⇒ ignores it
5. But disk now contains **partial real updates** with no committed log record explaining them

Now you have two problems:

- Disk may be inconsistent (half-applied metadata)
- Recovery *won't redo* (because no commit), and redo is what would normally "finish" the operation safely

✅ So you must ensure:

- **`TxEnd` is durable first**
- then Phase 3 writes can happen in any order, because recovery will redo if needed

That's what "TxEnd defines durability" means:

- Once `TxEnd` is on disk, the system promises: "this operation is committed and will survive crashes."
- The user-visible point where an operation becomes permanent is **when `TxEnd` is durable**, not when home blocks finish writing.

Phase 3 ordering: "can be lazy/asynchronous/reordered"

What Phase 3 does

Apply the logged updates to their "home locations":

- inode blocks
- directory blocks

- bitmaps
- (sometimes data blocks too, depending on journaling mode)

Why Phase 3 does NOT need strict synchronization

Because after Phase 2, we have:

- `TxEnd` is on disk ✅
- full redo info is on disk ✅

So if we crash anywhere during Phase 3, recovery logic is:

- sees `TxEnd` ⇒ redo transaction
- redo will re-apply any updates that didn't make it
- if some updates *did* make it, redoing them again is safe (idempotent overwrite)

That's why:

- Phase 3 can be delayed
- writes can be reordered
- crash during Phase 3 is OK

Important assumption: redo operations are designed to be **safe to replay** (writing the same final values again doesn't break consistency).

Phase 1 must be flushed before Phase 2
→ prevents "commit without redo info"

Phase 2 must be durable before Phase 3 matters
→ prevents "real updates happened but transaction looks uncommitted"

Phase 3 can be messy
→ because commit + redo log guarantee recovery can finish it

Logging: Crash Consistency

Case 1: Crash BEFORE `TxEnd` is written

State at crash:

- Log has `TxBegin` and maybe some operations
- NO `TxEnd`
- Filesystem disk state may be partially updated or not updated

Recovery rule:

Ignore the transaction

Why?

- No commit → transaction never logically happened

Result:

- Filesystem rolls back implicitly
- No replay

- Invariants preserved

Case 2: Crash AFTER `TxEnd` is written

State at crash:

- Log contains `TxBegin … TxEnd`
- Filesystem updates may be:
    - partially applied
    - fully applied
    - not applied at all

Recovery rule:

Redo the transaction

Why?

- Commit guarantees the operation must happen
- Replaying operations is safe (idempotent)

Logging: Recovery Process (Step-by-Step)

Input: System reboot after crash

Step 1: Scan the log

- Read log sequentially
- Identify transactions

Step 2: For each transaction

- If `TxEnd` **missing** → skip
- If `TxEnd` **present** → replay operations

Step 3: Apply operations

- Write inode blocks
- Write directory blocks
- Write bitmap blocks
- Overwrite is safe because redo is idempotent

📌 Recovery cost is **proportional to log size**, not disk size.

Redo Logging vs Undo Logging
Redo logging (what slides show)

- Log **new values**
- On crash:
    - committed → redo
    - uncommitted → ignore

Undo logging (mentioned briefly)

- Log **old values**
- On crash:
  - uncommitted → undo
  - committed → already applied

Modern filesystems mostly use **redo logging**.

Why garbage collection is needed

Because:

- Log is append-only
- Old committed transactions pile up

Cleanup process:

- Once transaction's operations are fully applied
- Log entries can be discarded or log wrapped around

This is **log cleaning**, not filesystem cleaning.

Logging vs FSCK (performance comparison)

FSCK

- Scans entire disk
- Time ∝ disk size
- Long reboot time
- Repairs consistency, not intent

Logging

- Scan only log
- Time ∝ log size (small)
- Very fast reboot
- Preserves user intent

📌 **Logging dominates fsck for modern systems**

---

1) Monolithic kernel

What it is

A monolithic kernel puts "the entire OS" inside the kernel: big pile of services (process mgmt, VM, FS, drivers, networking, …) all running at high privilege.

Key properties from the slides:

- "Everything executes at the higher privilege level" (e.g., supervisor mode)

Exports a high-level interface via system calls

Often supports loadable kernel modules (trusted extensions like drivers)

Monolithic end of the spectrum: huge syscall surface, single protection domain.

How it relates to the syscall interface

- Because the kernel *contains* most OS services, it tends to export lots of syscalls (Linux is the extreme; xv6 is “small” but still monolithic in structure).

How it relates to architecture / implementation

- Internal “calls” between subsystems are usually just function calls inside the kernel address space (fast).
- Downside: a bug in a driver is in the same privilege domain → can crash the whole machine / violate isolation.

What happens for `read()` (monolithic)

Think of a normal `read(fd, buf, n)`:

1. User code executes `read()` wrapper → triggers trap into kernel (syscall).
2. Kernel validates args + `fd` → finds the kernel “file object” for that fd.
3. Kernel goes into the filesystem layer (VFS / inode / page cache logic depending on OS).
4. If data is cached: copy kernel → user buffer, return.
5. If not cached: kernel calls the block layer + device driver in-kernel. Driver programs device / waits for interrupt.
6. Interrupt arrives → kernel interrupt handler wakes the blocked thread → finish copy → return.

Key point: inside monolithic kernels, most of that path stays inside one protection domain, so it’s mostly syscall entry/exit + (maybe) interrupt handling overhead, not IPC overhead.

---

2) Microkernel

What it is

A microkernel moves *most* OS services into user-space servers. The kernel keeps only the minimum needed to support an OS.

The slide picture explicitly shows user-space services like:

- Device driver server
- File system server
  …and the kernel still providing core mechanisms like IPC/scheduling/VM pieces.

The minimality principle (Liedtke)

> **“A concept is tolerated inside the microkernel only if moving it outside the kernel … would prevent the implementation of the system’s required functionality.”**

The slides also note reality: scheduling is typically still in-kernel even though “in theory” it could be exported.

(Why? Because scheduling interacts with interrupts/timers and needs to be very fast + trusted; exporting it often makes everything slower/hairier.)

How it relates to the syscall/interface

Microkernels typically export a smaller syscall set, heavily centered around:

- IPC primitives
- capability/object operations
- minimal thread/address-space management

OS NOTES

The seL4 example: system calls like message send/recv/yield, with other calls as wrappers.

Why IPC latency suddenly matters (microkernel vs monolithic)

Because microkernels push core services into other processes, you do *more round trips* that look like:

- client ↔ kernel ↔ server ↔ kernel ↔ client

So IPC is on the critical path of things like file reads, networking, driver calls, etc. That's why the slides hammer "why is IPC latency so important for microkernels?"

IPC design choices (from slides)

- Early microkernels: synchronous IPC (sender blocks; message copied directly into receiver buffer).

Later: add/shift toward asynchronous IPC.

Message structure evolved:

- short messages in registers (fast) but hardware-specific
- long messages removed; replaced by shared buffers / passing handles ("share, don't copy").

Capabilities (seL4 flavor)

The slides emphasize capabilities as an "unforgeable token" giving rights to kernel-managed objects (threads, address spaces, endpoints, IRQs, untyped memory, …).

Access rights can include read/write/grant, etc. This ties directly to security design principles: "if you don't have a capability, you can't do anything" and "all operations revolve around capability ownership (which the kernel checks)."

What happens for `read()` (microkernel)

A typical microkernel-style file read looks like this:

1. App calls `read(fd, buf, n)`.
2. Instead of "kernel does filesystem," the C library (or OS personality layer) sends an IPC request to the File System Server (or VFS server).
3. FS server checks permissions/metadata; if it needs disk:
4. FS server sends IPC to the Block Device Driver Server.
5. Driver server interacts with hardware (maybe via kernel-mediated mechanisms / shared memory / mapped device memory).
6. When IO completes (interrupt), the driver server notifies FS server (IPC).
7. FS server replies to app (IPC), possibly using shared memory to reduce copies.

So you pay:

- syscall/trap overhead for IPC
- context switches between processes
- message passing / copying unless optimized by shared memory

That's the core *performance trade*: stronger fault isolation & modularity vs more crossings.

"Blurred lines" (important!)

The slides explicitly point out that monolithic vs microkernel isn't a clean binary:

- Linux can do user-space drivers via UIO (userspace I/O)

Linux can do user-space filesystems via FUSE (implement `read/write/stat/...` in userspace). So: a "monolithic" kernel can still adopt microkernel-like placement for *some* subsystems.

---

3) Exokernel

What it is (as the slides frame it)

Exokernels basically say: stop having the kernel provide rich abstractions like "files" or "processes" *as the core thing*. Instead:

- Kernel focuses on multiplexing + protection
- Applications (or libraries linked to them) implement the abstractions themselves (VM policies, FS logic, IPC, drivers, …).

The slide quote captures the philosophy: lower-level primitives can be implemented more efficiently and give more freedom to higher levels.

How it relates to the interface (syscalls)

- The kernel interface becomes "resource-ish" and low-level: allocate/revoke/protect resources (CPU slices, physical pages, disk blocks, NIC queues).
- Instead of syscalls like "open/read," you might get syscalls like "give me these blocks / map this page / send on this NIC ring," plus protection checks.

Why this can be faster: End-to-End argument connection

The slides explicitly connect exokernels to the end-to-end argument: only the application truly knows what guarantees it needs; OS abstractions can sometimes be overhead or the wrong policy.

What happens for `read()` (exokernel)

There may not *be* a kernel `read()` in the classic sense.

Instead:

1. App (or its library OS) has a filesystem implementation.
2. It asks the kernel for access to disk blocks (with protection).
3. It schedules IO and manages caching/prefetch itself.
4. It copies/decodes data into its own buffers.

This can be insanely fast for specialized workloads—but you just moved complexity into apps, and you may lose "one standard interface" that everything expects.

---

4) Unikernel / Library OS

What it is (from slides)

A unikernel compiles the application together with a library OS, keeping only the parts needed in the final image.

Key property called out:

- Single address space / single protection domain.

Why performance can be good

Because you often eliminate:

- many syscall crossings
- many context switches between “kernel/user”
- lots of general-purpose code paths you don’t need

(But you’re trading away strong intra-OS isolation because everything is “one blob” in one domain.)

What happens for `read()` (unikernel)

- Often it’s just a function call into the library OS’s FS/network stack, which may directly talk to the hypervisor/device backend depending on deployment.
- Very short path → great latency, but weaker isolation inside the image.

---

5) Comparing the architectures (performance + security)

Performance (typical shape)

Monolithic: fastest general-case path for many OS services because kernel subsystems are function calls inside one domain; fewer crossings.

Microkernel: can be slower unless IPC is extremely optimized, because services become IPC + context switches; thus IPC latency is a first-class performance constraint.

Exokernel: can be fastest for specialized apps (kernel bypass style), because you remove generic overhead and policy; but pushes complexity to apps.

Unikernel: very fast in a VM/cloud-like environment for a single-purpose service; very short internal call paths.

The slides’ “kernel bypass networking” discussion is basically the same performance story: at extreme packet rates, the generic kernel networking stack becomes the bottleneck (syscalls, context switches, general-purpose overhead).

Security / isolation (typical shape)

- Monolithic: big trusted computing base (TCB); bugs in drivers/FS stack are “kernel bugs.”
- Microkernel: smaller kernel TCB; drivers/FS in user space improves fault isolation; capability systems can enforce strict access control (“complete mediation”).
- Syscalls are “high-level” in the sense that they talk about OS abstractions: files, processes, sockets, etc.
- Exokernel: kernel TCB can be tiny (mainly protection), but apps become huge and security depends on correctness of app/library OS code.
- Unikernel: small attack surface in the sense of “only what you included,” but little internal isolation (single domain).

---

1) Goals of Power Management (PM)

The three goals are simple to list, but the *why* and *how they conflict* is what matters.

Goal A: Save energy (battery life)

- Battery life is about **energy**, not instantaneous power.
- Core identity:
  Energy (J) = Power (W) × Time (s).
- That means you can save battery in two conceptually different ways:
  1. **Lower power while doing the same work** (run CPU slower/at lower voltage, turn devices off, etc.)
  2. **Finish the work faster** and go back to sleep sooner
     (sometimes called “race to idle”: run fast briefly, then idle deeply)

Those two can *conflict*: running faster might raise power, but if it reduces time enough, energy can still go down.

Goal B: Reduce energy at datacenter scale

Same physics, but the objective is cost + infrastructure:

- Cooling cost

- Power delivery limits
- Reliability (heat accelerates failure)
- Performance stability (thermal throttling)

Goal C: Reduce heat/noise, avoid thermal throttling

- Power becomes heat.
- Heat forces:
    - fans (noise, battery drain)
    - throttling (reduced frequency even if you "want" speed)
    - reliability issues

So PM is not just "battery"; it's also "don't melt the device and don't annoy users."

---

2) Devices – Processors: DVFS for utilization (P-states)

You asked: **How does DVFS support PM during utilization?**

What "during utilization" means

It means: **the CPU is not idle**. It is actively executing instructions.
So you cannot use "idle sleep states" to save power — instead you adjust **how efficiently** you compute.

DVFS = Dynamic Voltage and Frequency Scaling

CPU dynamic power is dominated by switching activity:

- transistors charging/discharging capacitances.
  A common simplified model is:

$P_{dynamic} \approx C \times V^2 \times f \times activity$

The slide summary compresses this to:
Power ~ Frequency × Voltage²

The important intuition:

- **Lowering frequency** reduces power roughly linearly.
- **Lowering voltage** reduces power quadratically (huge win).

- But voltage and frequency are coupled: to run reliably at a high frequency you often need a higher voltage.

So DVFS chooses an operating point:

- high V, high f (fast, hot
- low V, low f (slow, cool)

Governors (how OS decides the target)

Governors are policies mapping "observed load" → "frequency target":

- **performance**: basically peg high frequency for responsiveness.
- **powersave**: keep low frequency to reduce power.
- **ondemand**: jump up when load rises, drop when load falls (reactive).
- **schedutil**: uses scheduler/utilization signals to target some utilization (e.g., “try to keep CPU ~80% busy” rather than maxing out).

Why schedutil is conceptually nice:

- The scheduler already knows runnable tasks, sleep/wakeup behavior, etc.
- It can try to avoid oscillation (constantly switching frequencies) and match actual demand.

Why DVFS saves energy (not just power)

Energy depends on both power and time.
Lower freq may:

- reduce power
- but increase time

So DVFS works best when:

- you’re not latency-bound
- you don’t stretch time too much
- you can still “meet deadlines” (UI responsiveness, audio, etc.)

If you lower f too much:

- tasks take longer
- you might keep other devices awake longer (screen, radio)
- total energy might increase even if CPU power decreased

That’s why governors try to balance responsiveness and efficiency.

---

3) Processors: Idle states (C-states) and wakeup-latency tradeoff

You asked: **What is the tradeoff between power consumption and wakeup latency for idle states?**

What “idle” means in this context

CPU has nothing to do *right now*:

- no runnable tasks
- waiting for an interrupt (timer, IO completion, network packet, user input)

In that case, DVFS isn’t the big lever — **turning parts off** is.

C-states = progressively deeper “sleep”

Think of C-states as “how much of the CPU/platform do we shut down?”

- Shallow idle: stop executing, maybe clock-gate parts of the core.
- Deeper idle: shut down more clocks, PLLs, caches, even power-gate parts.

The tradeoff: deeper sleep saves more power but costs more to wake

Deeper C-state ⇒ lower idle power because more circuitry stops switching or is power-gated. But **deeper C-state ⇒ higher wakeup latency**, because to run again you must:

- re-enable clocks / PLL lock (PLL lock can take time)
- restore internal state
- possibly refill caches / TLBs
- bring up power rails safely

There's also **transition overhead**:

- energy cost to enter/exit
- time cost to enter/exit
  So if you only sleep for a very short time, deep sleep can be a net loss: you pay transition cost and you don't stay asleep long enough to "earn back" the savings.

How the OS decides

At a high level:

- predict expected idle duration (based on next timer event, workload history, etc.)
- pick the deepest state whose "break-even time" is below that duration.

So this is basically a prediction + optimization problem.

---

4) System-wide PM states: Sleep vs Hibernate (and why they differ)

You asked: **What are system-wide PM states and how do they differ?** System-wide means: not just CPU — the OS coordinates **all devices + memory + tasks**.

Sleep (Suspend-to-RAM)

What's happening mechanically:

1. **Freeze tasks** (stop scheduling user processes; they keep their memory).
2. Put CPU(s) into a deep idle state.
3. Suspend devices (most are powered down / quiesced).
4. Keep **RAM powered** (usually self-refresh) so the entire machine state (memory image) is preserved.

Result:

- wakeup is relatively fast (RAM already contains state)
- but there's still **standby power draw** (RAM refresh + some platform logic)

Hibernate (Suspend-to-Disk)

Mechanics:

1. Everything like sleep **plus**:
2. OS writes the entire memory image to nonvolatile storage (disk/SSD).
3. Then RAM can be powered off completely.

Result:

- much lower standby power (near off)
- but suspend/resume are slower due to disk IO (write image on entry; read image on resume)

The key conceptual difference

- Sleep preserves state in **RAM** (fast wake, some drain).
- Hibernate preserves state in **disk** (slow wake, minimal drain).

---

5) Drowsy Power Management: Key ideas in depth

You asked for **Drowsy PM (key ideas)** with detail.

The problem Drowsy targets: "background wakeups"

Phones and mobile systems wake frequently for small tasks:

- periodic sync
- push notifications
- background fetch
- sensor-related work
- network housekeeping

Individually, each event is short. But the system may:

- wake the whole phone
- spin up devices
- run services
- then go back to sleep

The critical observation: transitions dominate

For these short events, the "useful work" may be tiny compared to the overhead of:

- waking CPUs
- resuming drivers
- starting system services
- re-enabling devices
- reestablishing radio states

That's why the lecture emphasizes that a huge fraction of energy is in **transitions** (the slide gives the dramatic stat).

So the win condition is:

- **reduce how often you do full transitions**, or
- **reduce how "big" each transition is**.

Android wakelocks (why they matter)

In Android's model:

- A background component holds a **wakelock** while it needs the system "awake enough" to run.
- When *all* wakelocks are released, the system is allowed to go to suspend.

So typical background event pattern:

1. wake triggered (timer/alarm/interrupt)
2. app/service acquires wakelock
3. does work (often involving network + system services)
4. releases wakelock
5. system suspends again

This is good because it prevents sleeping mid-task, but it also means frequent wakeups can keep causing expensive full resume/suspend cycles.

What "normal suspend/resume" does (and why it's expensive)

OS NOTES

Normal suspend is “global”:

- flush / sync buffers
- freeze tasks
- suspend devices (lots of drivers)
- disable CPUs
- RAM self-refresh

Normal resume reverses that:

- enable CPUs
- resume devices
- thaw tasks

If your background event only needs:

- alarm/timer
- WiFi
- a specific service/app
  …then resuming *everything* is wasted energy.

Drowsy’s core idea: partial wake

Instead of a binary world:

- fully on
- fully suspended

Drowsy introduces an intermediate mode:

- wake only the **minimum** set of tasks + devices needed for the current event
  (a “minimal wake set”)

Everything else remains suspended/frozen.

What is the “wake set” really?

It’s the closure of dependencies required for progress.

Concretely:

- If App A needs to run, but it blocks waiting for Service B → B must be awake too.
- If B needs device driver D (e.g., WiFi) → D must be awake.
- If D needs kernel mechanisms and certain low-level support → include those.

So the wake set is:

- **Tasks/processes/threads** that must run
- **Devices** those tasks must touch
- Plus any supporting system services in the dependency chain

“No modification to user-space” — why it’s hard

If you could rewrite apps, you could ask them: “tell me what devices you’ll need.”
But you can’t.

So Drowsy has to *infer* needs by observing behavior.

How Drowsy expands on-demand (important subtlety)

You can't always know the full wake set up front.

So Drowsy does:

1. Start with a minimal predicted set (alarm + some known services).
2. As code runs, if it tries to touch a suspended resource (file/device/socket/etc.),
3. the system detects it and **resumes just that dependency**,
4. then continues.

This makes it robust:

- if your prediction was incomplete, you still don't break correctness
- you just "pay" a small incremental resume cost rather than resuming everything

How it detects dependencies (conceptually)

The lecture shows instrumentation/wrapping:

- intercept key operations that imply dependencies:
    - file ops
    - networking
    - device control interfaces (ioctl)
    - other kernel/user boundaries that indicate "you need X"

When the wrapper sees "operation requires device that is currently suspended," it:

- resumes that device
- then lets the operation proceed

This is basically **lazy dependency activation**.

Why this helps energy

Because the expensive part is often "resume the whole world."
Drowsy tries to keep the wakeup "surgical":

- fewer devices resumed
- fewer tasks thawed
- fewer drivers reinitialized
- less transition work overall

So you still handle background events, but you pay closer to:

- "wake only what you needed"
  instead of:
- "wake everything because something happened."

PART I — Scheduling (CFS → EEVDF)

---

1. What problem is CPU scheduling *actually* solving?

OS NOTES

At the most fundamental level:

> **One CPU, many runnable tasks**
> → decide **who runs next**, **for how long**, and **with what fairness guarantees**

Schedulers must balance:

- **Fairness** (everyone gets their share)
- **Responsiveness** (interactive tasks wake fast)
- **Throughput** (keep CPU busy)
- **Low overhead** (scheduler itself shouldn't dominate CPU)
- **No starvation**

Linux's scheduler evolution reflects *changing priorities* in these goals.

---

2. Refresher: Completely Fair Scheduler (CFS)

Core idea

CFS tries to simulate **ideal multitasking**:

> *If N tasks are runnable, each should get 1/N of the CPU over time.*

But CPUs run tasks **sequentially**, so Linux tracks *virtual runtime*.

Virtual Runtime (vruntime)

Each task has: task.vruntime = how much CPU it has received (scaled by priority)

Higher priority (lower nice value) → vruntime increases **more slowly**.

From the slide:

A.vruntime += t * (weight[20] / weight[20 + priority])

Interpretation:

- Two tasks run for the same **real time**
- Higher priority task looks like it ran **less** in virtual time
- Scheduler favors tasks with **small vruntime**

Wake-up problem (important!)

If a task sleeps for a long time:

- Its vruntime does **not** increase
- When it wakes up, it looks like it's been starved
- It could monopolize the CPU

CFS fix:

> When a task wakes up, set its vruntime to the **minimum vruntime** in the runqueue

This prevents “sleep to cheat” behavior.

How long does CFS run a task?

Time slice:

```
max(min_granularity, sched_latency / #tasks)
```

- `sched_latency`: target time for all tasks to run once
- `min_granularity`: prevents extremely tiny slices

Key limitation of CFS

CFS answers:

> “Who has run the least so far?”

But it **does NOT directly model latency guarantees** or short requests.

This becomes a problem for:

- Interactive tasks
- I/O-heavy tasks
- Bursty workloads

---

3. Why Linux moved to EEVDF (Earliest Eligible Virtual Deadline First)

Core motivation:

> CFS approximates fairness, but does not model *service deadlines*.

EEVDF brings ideas from **real-time scheduling** into Linux fairness.

---

4. What EEVDF is trying to solve

EEVDF tries to answer **two questions simultaneously**:

1. **Who deserves CPU time right now?** (fairness)
2. **Who should finish their request earliest?** (latency)

CFS only answers (1).

---

5. EEVDF vs CFS — Conceptual differences

Same ideas:

- Proportional sharing
- Weight-based fairness
- Virtual time

New ideas:

- Tasks specify **request lengths**
- Scheduler tracks:
    - **Lag**
    - **Virtual deadlines**

This lets Linux distinguish:

> "This task is behind" vs
> "This task must finish *soon*"

---

6. Resource share (formal definition)

For task *i* at time *t*:

$f_i(t) = w_i / \Sigma w_j$ (over runnable tasks)

Meaning:

- Task with higher weight deserves a larger fraction of CPU

Ideal service over interval $[t_0, t_1]$:

$S_i(t_0, t_1) = \int f_i(\tau)\, d\tau$

But real execution ≠ ideal execution → we need error tracking.

7. Lag — measuring unfairness

Lag measures:

> “How much CPU a task *should* have gotten minus how much it *actually* got”

lag_i(t) = ideal_service − actual_service

Interpretation:

- `lag > 0` → task is **owed** CPU
- `lag < 0` → task has had **too much**

8. Eligibility

EEVDF rule:

> A task is **eligible** only if its lag is positive

This prevents:

- Tasks that already over-ran from jumping the queue

9. Virtual Time (V(t))

EEVDF defines:

V(t) = ∫ (1 / Σ w_j) dt

Meaning:

- Virtual time advances **faster** when fewer tasks exist
- Each task consumes `w_i` real time per unit of virtual time

This normalizes fairness.

10. Virtual deadlines (key idea)

Each task has a request of length `r`.

Virtual deadline:

V(d) = V(e) + r / w_i

Interpretation:

- Smaller request → earlier deadline
- Higher weight → earlier deadline
- Combines fairness + latency

11. Scheduling rule

> **Among eligible tasks, run the one with the earliest virtual deadline**

This is the heart of EEVDF.

---

1) How are device registers mapped via virtual memory for MMIO?

The core idea: "device registers live at addresses"

Many devices expose a set of **control/status/data registers**. In modern systems these registers are often **memory-mapped**:

- The device is assigned a **physical address range**: `<Start, Len>`
- When the CPU executes a normal `load` or `store` to that address range, the bus routes it to the device (not to RAM)

So MMIO is literally:

> **device registers are mapped into the CPU's address space**.

Why virtual memory matters

Your code (kernel or sometimes user code) uses **virtual addresses (VA)**. Hardware accesses **physical addresses (PA)**. So for the CPU to reach device MMIO registers, the OS must ensure:

> the device's PA range is **mapped** in the current page table, so those VAs translate to the device PA addresses.

In xv6, the slides explicitly point out that devices have fixed physical locations and are mapped by the kernel page table (in `kvminit()`), e.g., PLIC and UART0

Concrete slide example: PLIC mapping

The deck shows the PLIC device has:

- `<Start, Len> = <0x0C000000, 0x400000>`

and that xv6 maps it in `kvminit()` (via a `kvmmap(...)` call) so the kernel can read/write PLIC registers through MMIO

Concrete slide example: UART0 mapping

Similarly UART0 has:

- `<Start, Len> = <0x10000000, 0x1000>`

and is also mapped by the kernel page table so the UART driver can do `ReadReg/WriteReg` into that region

.

What the code looks like conceptually

Drivers typically do something like:

- "Read status register"
- "Write transmit register"
- "Read receive register"

The slides show this pattern in the UART example with:

- “Load 0x10000005”
- “Store 0x10000000”
  corresponding to accessing **UART registers at offsets** inside the UART MMIO window

Key mental model:
A “register” is implemented as “a memory location that hardware intercepts.”

---

2) How are devices connected to the CPU?

Base system model (slides)

The slides start with the OS model:

- CPU + RAM + Device on a **system bus**
- CPU communicates via **loads/stores**
- Device signals back via **interrupts**

So:
CPU → device: memory operations (MMIO)
device → CPU: interrupts

End-to-end picture (apps → driver → CPU → bus → device)

The slides emphasize the “end-to-end path” using UART:

1. **App** calls something high-level (`printf`)
2. That triggers `write(1, ...)` to stdout
3. Kernel routes that to console code
4. Console code calls the UART driver interface (`uartputc`, etc.)

UART driver performs **MMIO loads/stores** to UART registers on the bus

UART hardware toggles voltages on the TX wire to send bits to the console

That is the complete “software call chain → hardware side effect” the deck wants you to understand.

What is the role of the peripheral bus?

The slides distinguish:

- Devices directly on the **system bus** (common in SoCs/embedded)
- Devices behind a **general/peripheral bus** (PCIe, I2C, SATA, etc.)

A peripheral bus exists because:

- It can support many devices
- It has its own protocol/wires/transactions
- It often needs **addressing/IDs** to select which device you’re talking to

Example given: I2C (SDA/SCL, device IDs) , and UART as a simpler bus with RX/TX

What does the bus controller do? How does software interact with it?

From the slides:

Bus controller’s job

OS NOTES

A bus controller:

- “manages the bus interconnect”
- “translates higher-level ops (e.g., MMIO) to lower-level protocol-specific wire operations”

So the controller is the “bridge” between:

- **CPU-side** view: “read/write these MMIO registers”
- **bus-side** reality: “send the right signaling on PCIe/I2C/SATA wires”

How software interacts with the controller

Software interacts via **MMIO registers of the controller** (or device window managed by the controller). The CPU just does normal loads/stores to those mapped addresses. The controller then performs the physical bus transactions.

That’s why the deck says devices may be mapped as `<Start, Len>` regions: the controller/device exposes registers in an MMIO region

---

3) What patterns are used in software to interact with devices? (Polling, Interrupts)

The slides explicitly call out **two common patterns**

Pattern A: Polling

What polling is

Polling means:

- repeatedly read a **status register**
- loop until it changes (e.g., BUSY → READY)

Slides show canonical pseudo-code:

while (read(DEV_STATUS) == BUSY) { }

write(DEV_DATA, ___)

write(DEV_COMMAND, ___)

while (read(DEV_STATUS) == BUSY) { }

Why this works

The device exposes “am I ready?” via a status bit.
The CPU just keeps checking.

Scheduling / utilization implications

Polling has a huge OS consequence:

- If a thread is polling, it is **running** (consuming CPU time) even though it’s “waiting for hardware.”
- That wastes CPU cycles and can starve other runnable work.
- On a single core, polling can destroy system responsiveness because it keeps the CPU busy doing nothing useful.

The slides highlight that polling vs interrupts has “scheduling and utilization implications”

How you should say it in OS terms:

- Polling turns I/O wait time into **CPU busy-wait**.
- That increases **CPU utilization** but decreases **useful utilization**.
- Scheduler sees a runnable CPU-bound task, not a blocked task.

Pattern B: Interrupt-driven I/O

What interrupts change

Instead of the CPU repeatedly checking, the device tells the CPU:

> "I'm ready now" / "operation completed" / "new input arrived"

The slide shows a conceptual approach:

- `wait(cv, read(DEV_STATUS) != BUSY)`
- then write command/data
- then wait again

And an interrupt handler that does:

- `signal(cv)` when status changes

(Or a variant where the handler drains ready work in a loop.)

Scheduling / utilization implications

Interrupt-driven I/O:

- Lets the waiting thread **sleep/block**
- Scheduler can run **other tasks**
- CPU time is spent on useful work until the interrupt arrives

Costs:

- Interrupt handling overhead (trap, context save/restore, handler work)
- If interrupts are too frequent → overhead and possible "interrupt storm"

So the tradeoff is:

- **Polling:** low latency but wastes CPU
- **Interrupts:** efficient CPU usage but added overhead and complexity

---

4) How are interrupts supported?

Hardware model from slides

Slides state:

- Devices communicate with CPU via interrupts

A PLIC manages external interrupts for all cores/harts. OS interacts with PLIC via MMIO registers

---

(a) How does the kernel interact with interrupt controller (PLIC) to determine the interrupt #?

What PLIC does

PLIC collects interrupt requests from devices (UART, disk, etc.) and then, per-hart:

- decides which interrupt is pending and allowed (enabled/priority)
- presents an interrupt to the CPU
- allows the OS to “claim” the interrupt number

Slides give concrete register grouping:

- Enable registers are grouped per hart (e.g. Hart 0 enable region)
- Claim register has a fixed offset (e.g. Claim Hart0 = Start+0x201004)

The kernel’s steps (conceptually)

When an external interrupt trap occurs:

1. Trap handler recognizes “external interrupt”
2. Kernel calls something like `plic_claim()` to read the PLIC claim register
3. That returns an **interrupt ID (irq number)**

The slides show this exact claim/complete flow and connect it to xv6 `trap.c` and `plic.c`

(b) How does the OS invoke the relevant device driver handler? (e.g., uartintr)

The deck says: in xv6, external interrupts come from:

- UART0 (input chars / keystrokes)
- VIRTIO0 disk (I/O completion)

So once the kernel has the IRQ number from PLIC:

- if it matches UART’s IRQ → call `uartintr()`
- if it matches VIRTIO’s IRQ → call disk’s interrupt handler

This is precisely what the “Interrupt Handling via PLIC” slide is illustrating: PLIC gives you an `irq`, and the trap code chooses the right handler

Important OS concept:
This is *interrupt demultiplexing*: one entry point (trap) → choose correct device handler.

---

5) What is DMA? When/why is it useful?

The problem: large transfers are expensive for CPU

Slides say: for large transfers (disk blocks, framebuffers, packets), without special support:

- CPU must spend cycles copying memory to/from the device

That means:

- CPU becomes the bottleneck
- extra cache pollution
- more time in kernel copying buffers

DMA definition

DMA = **Direct Memory Access**:

a hardware engine copies between memory and device without CPU doing the copy loop.

The slide calls it “offload to hardware” and introduces a DMA engine that reads/writes memory without CPU involvement

---

How DMA works (host-to-device example in slides)

The deck’s sequence (writing to disk):

1. CPU writes DMA configuration registers: (from, to, length)
2. CPU writes DMA command register to start transfer
3. DMA engine does the copy
4. DMA raises interrupt on completion

So DMA is almost always paired with interrupts:

- CPU initiates
- CPU goes away (runs other tasks)
- completion interrupt wakes up the waiting code path

When/why DMA is useful (what you should say)

DMA is useful when:

- transfers are **large** or frequent
- you want to reduce CPU overhead
- you want higher throughput (disk/network/GPU)

Why it matters for scheduling:

- Without DMA: the thread may burn CPU doing copy work → looks CPU-bound
- With DMA: the thread can block waiting for completion interrupt → scheduler can run others

---

Quick “exam-style” answers list (your exact questions)

Q: How are device registers mapped via virtual memory for MMIO?

A: Devices expose register address ranges `<Start,Len>`; the kernel maps those physical ranges into its virtual address space via page tables (xv6 does this in `kvminit()`), so loads/stores to those VAs translate to the device PA and are routed to hardware. See PLIC and UART0 examples in the deck

Q: How are devices connected to the CPU? End-to-end picture?

A: Apps call syscalls → kernel drivers → CPU executes loads/stores to MMIO registers over the system/peripheral bus → bus/controller routes transaction to device → device signals CPU with interrupts. UART end-to-end is explicitly shown

Q: Role of peripheral bus?

A: Provides a shared interconnect for multiple devices, often requiring addressing/IDs and protocol-specific signaling (examples: PCIe/I2C/SATA).

Q: What does the bus controller do? How does software interact?

A: Controller manages bus and translates CPU-visible MMIO reads/writes into the bus’s wire protocol operations; software interacts by reading/writing the controller/device’s MMIO-mapped registers.

Q: Polling vs Interrupts patterns + scheduling/utilization implications?

A: Polling busy-waits on status registers (wastes CPU, harms scheduling). Interrupts let the thread sleep and get woken by device, improving CPU utilization but adding interrupt overhead. Patterns shown in slides

Q: How are interrupts supported? How OS gets interrupt #? How dispatches to handler?

A: External interrupts are managed by PLIC; kernel MMIO-reads claim register to get `irq`, uses it to pick handler (`uartintr` for UART, disk handler for virtio), then MMIO-writes complete to PLIC. Slides tie this to xv6 `trap.c` and `plic.c`

Q: What is DMA and when/why useful?

A: DMA offloads bulk copying from CPU to hardware engine; CPU programs DMA registers then waits for completion interrupt. Useful for large transfers (disk blocks, packets, framebuffers) to reduce CPU cycles and improve throughput

Kernel dispatches based on that ID

Kernel calls `plic_complete(irq)` to tell PLIC it's handled

Why DMA helps even if you already use interrupts

Interrupt-driven I/O alone fixes *waiting*, not *copying*

Interrupts mainly solve the "don't waste CPU cycles polling" problem:

- With polling: CPU spins checking a status register → wastes CPU.
- With interrupts: CPU sleeps / runs other tasks, device interrupts when ready.

But even with interrupts, if the I/O involves **large data** (disk blocks, network packets, GPU framebuffers), the CPU might still have to do:

- copy user buffer → kernel buffer
- copy kernel buffer → device FIFO/register window
- or copy device buffer → kernel → user

That copying can dominate runtime.

DMA removes that CPU copying work: the device (or DMA engine) transfers bytes directly between device and RAM, and then triggers an interrupt when done.

So the CPU goes from doing:

- **O(bytes)** work per I/O (copy loops)
  to:
- **O(1)** work per I/O (program descriptors/registers + handle completion)

When it’s especially helpful

DMA is a big win when transfers are:

- **Large** (KB/MB)
- **Frequent** (high packet rate, fast storage)
- **Streaming** (audio/video/network)
- **Parallelizable** (multiple outstanding requests)

Examples:

- Disk: read/write 4KB blocks and up
- Network: RX/TX rings with many packets
- GPU: transferring buffers / frame updates

But DMA is not always worth it

DMA has overheads and complications:

1) Setup overhead

To do DMA you must:

- allocate buffers
- provide physical addresses (or IOMMU mappings)
- build descriptor tables / program DMA registers
- manage completion

For **tiny transfers** (like a single character on UART), DMA setup can cost more than just doing programmed I/O.

2) Memory constraints: pinning and mapping

DMA needs stable memory locations:

- pages may need to be **pinned** (can’t be swapped/moved)
- may need **bounce buffers** if device can’t reach all RAM
- IOMMU may be required for safe flexible mappings

3) Cache coherency issues

CPU caches and DMA writes can disagree unless the platform is coherent or the OS flushes/invalidates caches correctly.

4) Security/isolation

A DMA-capable device can potentially read/write arbitrary RAM unless constrained (IOMMU). So DMA increases the importance of protection mechanisms.

---

1) What “security” means in OS class terms

The “security mindset”

The lecture’s point is: don’t look at a system as “what it was intended to do.” Look at it like an attacker: **what assumptions can I violate, what pieces can I combine, what “harmless” feature becomes dangerous when composed with another?**

That’s why the slide emphasizes that *seemingly harmless exploits can combine to become serious.* In OS terms: a tiny info leak + a tiny privilege mistake + a race condition can become full compromise.

---

2) Security engineering framework: Goals → Threat Model → Mechanisms

The lecture frames security design as “three easy pieces”:

1. **Security goals / policy** (what guarantee do we want?)
2. **Threat model** (what do we assume about attacker + trusted parts?)
3. **Mechanisms** (what tools enforce policy under that threat model?)

2.1 Security goals / policy (what you want)

They list common goals:

- **Confidentiality:** prevent unauthorized disclosure
- **Integrity:** prevent unauthorized modification
- **Availability:** ensure service/data access for authorized users
- **Authenticity:** prevent forging origin of data/action
- **Non-repudiation:** prevent denying an action

OS translation: these goals apply to *memory*, *CPU*, *files*, *devices*, *IPC*, etc. (so policies get complicated).

2.2 Threat model (what you assume about attacker + environment)

A threat model answers:

- what can be compromised?
- what is trusted?
- what can/can't the attacker do?

Key idea: without a threat model, statements like "this is secure" are meaningless—because security is always conditional on assumptions.

The lecture also stresses "assumptions break in the real world," so we don't give up—we **raise the bar** and make attacks more expensive / less rewarding.

2.3 Mechanisms (how you enforce)

Mechanisms are concrete tools: hashing, ACLs, capabilities, enclaves/TEE, etc. The lectures explicitly organize around:

- **Authentication** via cryptographic hashes
- **Authorization** via ACLs/capabilities
- **Trusted hardware** via SGX/TPM/TEE style ideas

---

3) How vulnerabilities "happen" in systems

They call out two major sources:

1. **spec/goal flaws** ("we asked for the wrong behavior" or forgot cases),
2. **implementation bugs** (deviations from desired behavior),
   plus **invalid assumptions** about environment/attackers (often "human element").

So security is hard because you need "deep understanding of systems at all layers."

---

4) Design principles (Saltzer & Schroeder style) and what they mean in OS

The slide list is the classic secure design principles. Here's what each means *in OS terms*:

Economy of mechanism = keep it simple

Simpler designs have fewer bugs and fewer "weird corners" attackers can use. In OS: smaller kernel, fewer special cases, less duplicated logic.

Fail-safe defaults = deny by default

Default should be "no access unless explicitly allowed." That's **allow-list** thinking. In OS: a process should not read random memory/files unless given rights.

Complete mediation = check every access

Every time someone tries to access a resource, the system must re-check authorization (not “checked once earlier so we assume it’s fine forever”). In OS: permissions checks can’t be bypassable with caching mistakes.

Open design = don’t rely on obscurity

Security shouldn’t require “the attacker doesn’t know how it works.” OS kernels are open-source and still secure by design.

Separation of privilege = multiple conditions

E.g., 2FA is the classic example. In OS: maybe you need both “is admin” and “is on console” or “has capability X + correct label.”

Least privilege = give minimum rights needed

Very OS-core: each component should run with just the permissions it needs. “Root everywhere” is bad.

Least common mechanism = reduce shared mechanisms

Shared mechanisms are shared attack surfaces. In OS: avoid global shared structures that leak between users/processes.

Psychological acceptability = usable security

If it’s painful, users will bypass it (“chmod 777 everything”), which defeats security.

Extra ones:

- **Work factor**: compare attacker cost vs payoff/resources
- **Compromise recording**: logging/auditing helps when things go wrong

---

5) Trusted Computing Base (TCB) and “reducing what must be trusted”

TCB definition (as used here): total protection mechanisms enforcing the policy (hw/firmware/software). Bigger TCB ⇒ more places bugs compromise security.

They list two major reduction approaches:

5.1 Decomposition + isolation (microkernel idea)

A monolithic kernel has *everything* in the kernel (VM, drivers, FS, scheduling, etc.). A microkernel pushes big subsystems (drivers/FS) out to user space, so kernel is smaller and easier to trust.

5.2 Formal verification (seL4 idea)

You write a spec, list environmental assumptions, and prove the implementation meets the spec (given assumptions). This reduces “trust by hope.”

---

6) Authentication: passwords + cryptographic hash functions

6.1 What authentication is (in OS setting)

Authentication = verifying identity of the principal performing an action. In an OS, processes must be associated with user identities (UID/GID etc.).

6.2 Password system flow (the exact OS story)

The lecture gives a concrete login flow:

1. user types password into `/bin/login`
2. OS computes `hPWInput = H(password)`
3. retrieves stored `<hPW, UID, ...>` from storage (example mentioned: `/etc/passwd`)
4. compares `hPWInput` to stored `hPW`
5. on success: `fork()`, set up fds, `setuid(UID)`, `setgid(GID)`, `exec` the user shell

Why this matters: OS isn’t storing the password in plaintext. It stores *a hash*, and checks whether the user knows the password by hashing what they typed and comparing.

6.3 Hash properties you’re expected to know

They define a cryptographic hash function as `y = H(x)` mapping arbitrary-length input to fixed-length output and list the standard properties:

- pre-image resistance
- second pre-image resistance
- collision resistance
- avalanche effect

6.4 “If attacker steals password hashes, are we safe?”

The slide asks this explicitly. The key reasoning:

- Pre-image resistance says “hard to find *some* input for an arbitrary hash output,”
- but in password attacks, the attacker doesn’t need to invert a random hash; they do **dictionary/brute-force guessing**: try likely passwords, hash them, compare to stolen hashes.
  So stolen hashes are dangerous because user passwords are often low-entropy.

(Even if the slide doesn't say "dictionary attack" explicitly, this is exactly what it's pointing you to conclude from "attacker learns hashes—are we safe?")

---

7) Authorization: reference monitor + policies + ACLs vs capabilities

7.1 Authorization definition

Authorization = verifying whether a principal can perform an action, based on a security policy.

7.2 OS story: subject → operation → object, checked by reference monitor

Concrete example in slides: a user is logged in; a process (bound to UID) calls `read()` on some file path. The OS must decide allow/deny using a **Reference Monitor**.

Important mental model:
Every security-sensitive operation funnels through an enforcement point (reference monitor), which consults policy.

7.3 Access Control Matrix (conceptual model)

They show an access control matrix: rows = subjects (UIDs), columns = objects (files), entries = rights (R, W, etc.).

You rarely store the full matrix explicitly; you implement it in two main ways:

7.4 ACLs vs Capabilities (core compare/contrast)

The slides explain both:

ACL (Access Control List) = "column oriented"

- stored with the **object**
- maps "which subjects can do what on this object"
- analogy: bouncer checking names

Capability = "row oriented"

- stored with the **subject**
- an **unforgeable token** granting access to an object for certain operations
- analogy: a key to a door

7.5 The three exam-style questions the slides ask

The slides basically want you to be able to reason about operational consequences:

(A) If you change who can access an object, what happens?

- **ACLs:** easy: edit the object's ACL (add/remove subjects) because the object stores the list.

Capabilities: harder because rights are distributed among subjects; revocation often requires extra machinery (indirection, versioning, or tracking issued caps) since you can't just "edit one list" unless you built the system that way.

(B) If you want to enumerate what a subject can access, what happens?

- **Capabilities:** easy: subject's capability set *is the list* of what it can access.

ACLs: harder: you'd have to scan many objects' ACLs to find entries for that subject.

(C) Delegating limited access to a child process (least privilege)

The slide asks: which better supports least privilege delegation?

- **Capabilities** naturally support handing a subset of rights to a child: "give the child this one key, not your whole identity's power."
- **ACLs** tend to be more identity-based: child's access is often derived from "who it runs as" (UID), so least-privilege delegation can be clunkier unless you create new identities/roles or add special mechanisms.

---

8) Stronger app security via trusted hardware: enclaves (Intel SGX idea)

8.1 Problem: large TCB for applications

If the OS is compromised, app confidentiality/integrity can be violated—because the OS is in the TCB for most app protections.

8.2 Enclave / TEE concept

An **enclave** is an isolated region in an application's address space, designed so code/data inside get confidentiality + integrity protection even if the OS is untrusted. The slide uses Intel SGX as the example.

Key picture idea: app is "untrusted" overall, enclave part is "trusted"; OS is outside the enclave trust boundary.

8.3 "Authentication for software principals"

They ask: how do we identify a program? The goal is remote attestation / secure remote execution: you run code on an untrusted machine owner and still want to verify *what program actually ran.*

---

9) Side-channels & speculative execution: what the OS threat model must include

Lecture 21 brings in "side-channels" as attacks arising from *implementation on real machines* (cache/timing, etc.).

They refresh:

- **Rowhammer** as a hardware-level reliability→security issue

Speculative execution as performance optimization that creates leakage channels

And they outline:

- **Meltdown:** speculatively access privileged memory before exception is handled
- **Spectre:** train branch predictor to speculatively execute out-of-bounds/sandbox escapes and then leak via microarchitectural state (like cache timing).

---

10) Mitigations: software patterns (retpoline, KPTI) and the performance cost

Retpoline (Spectre v2 / branch target injection)

The slide describes retpoline as a mitigation that “traps” incorrect speculation into a safe loop, using the return stack buffer (RSB).

KPTI (Meltdown)

Kernel Page Table Isolation: user page tables contain only minimal kernel entry points; kernel mappings are separated so user-mode can’t speculatively see kernel memory the same way. But it adds overhead because you effectively need more costly transitions.

Big theme: mitigations cost performance

They emphasize that mitigations add overhead and ask: how do we enable them only when needed?

---

11) USC (Unmapped Speculation Contract): a precise HW/SW contract

11.1 Informal idea

USC says (informally): if a physical page is **not referenced by active page tables / TLB entries**, speculation should not be able to access data in that page. This gives software a clean rule for what speculation may touch.

11.2 Why the lecture likes contracts

They argue many past “informal contracts” were wrong (Spectre/Meltdown violated assumptions like “speculation doesn’t matter,” Rowhammer violated assumptions about isolation). USC is helpful because it’s:

- precise HW/SW contract
- implementable efficiently

- doesn’t over-specify
- gives software control
- bridges semantic gap between HW and SW

11.3 Formal flavor (what you need to recognize)

They mention a noninterference/nonleakage style formalization: two states indistinguishable to adversary remain indistinguishable after executing traces/steps.

You don’t necessarily need to reproduce the formalism, but you **do** need the idea: USC tries to guarantee that speculation can’t be influenced by unmapped secrets.

---

12) WARD design: splitting “public” vs “sensitive” mappings (Q/K domains)

WARD is presented as a design that leverages USC.

12.1 Two domains

They show: **Q domain** contains public + per-process data; **K domain** contains sensitive data (“all mappings”).

The diagram on that slide (page 24) is basically: each process has structured memory regions and WARD tries to ensure sensitive mappings are isolated from what can be speculated-on in a dangerous way.

They pose key design questions:

- why does K still require software mitigations?
- problems with sharing stack between Q and K?
- how to handle text segment addresses?

12.2 OS implementation implication: split process state

WARD changes process management: split `struct proc` into:

- **Sensitive part:** saved CPU context, e.g., trapframe in xv6
- **Public part:** PID, scheduler metadata, etc.

And they mention scheduling changes:

- scheduler can choose next thread without world switch
- but if switching to a different process, a world switch is needed in context switch

12.3 Performance result

They include a performance chart comparing “Linux-style mitigations” vs “USC-based mitigations” for WARD, indicating USC-based approach can reduce overhead in many operations

OS NOTES